GRS

PLAN SQUELETTE MAÎTRE — v1.1 — PERFECTION

Core d'entreprise autonome, clé en main, installable dans une infrastructure IA
10 modules M — LES 10 PLANS MAÎTRES INTÉGRÉS — Passes 01-05 + méthode d’exécution PROOF STACK v1.1 intégrée

Doctrine : preuve avant déclaration — aucun « fake done », aucun stub
Premier module en développement : M7 Marketing & Croissance IA (D-008)

14 juillet 2026 — Saint-Rémi, Québec
Statut : 10/10 plans reçus — Passe PERFECTION appliquée : 3 recherches + 4 simulations internes -> 16 correctifs (C-01..C-16) + calibration METR — PRÊT POUR EXÉCUTION

1. Vision et doctrine

GRS donne à une PME les capacités opérationnelles d'une grande entreprise en unifiant les données fragmentées (réception, CRM, estimation, terrain, comptabilité, inventaire, marketing, analytique) en gestion actionnable et automatisée. **GRS est un core autonome et clé en main** : il ne dépend de rien, embarque ses logiciels spécialisés, et a une seule obligation — être installé dans une infrastructure IA et s'y brancher (noeuds, modèles routés, RAG) via le dashboard. Première cible : l'entreprise de rénovation extérieure de Nathan. Long terme : SaaS PME québécoises (Loi 25 rehaussée).

1.1 Règles non négociables

- **REG-01 — Preuve avant déclaration.** Rien n'est « fait » sans preuve exécutable au ledger. « Documenté = fait » = défaillance critique.
- **REG-02 — Aucun fake done, aucun stub.** Stub, TODO silencieux, mock présenté comme fonctionnel = REJECTED automatique.
- **REG-03 — REJ-0040.** Aucun score global opaque. Verdicts APPROVED / REJECTED par dimension, avec preuves.
- **REG-04 — CIV : jamais auteur et juge.** Siège de validateur tournant si le validateur désigné est l'auteur.
- **REG-05 — Indépendance critique des modèles.** Revues en aveugle, artefacts anonymisés, aucune influence croisée avant dépôt.
- **REG-06 — Un module M à la fois.** Terminé, testé, re-testé, prouvé 100 % fonctionnel avant d'ouvrir le suivant.
- **REG-07 — Le contexte est une ressource précieuse.** Économie maximale de tokens ; mémoire longue = RAG + MCP + ledger.
- **REG-08 — Briques embarquées, jamais figées.** Intégrées DANS le core ; toute brique nommée = candidate ; la meilleure surclasse automatiquement via le pipeline Q1-Q6 ; wrapper d'interface = remplacement transparent.
- **REG-09 — Sécurité par défaut.** Secrets chiffrés, exposition minimale, autorisation fine, journaux d'audit.
- **REG-10 — Conseils fiscaux/comptables encadrés.** Mention obligatoire de consulter un professionnel (CPA/fiscaliste), codée en dur dans M5.
- **REG-11 — Dégradation gracieuse.** Infra sous le minimum = mode bridé : le temps réel bascule en horaire/planifié. GRS fonctionne toujours.
- **REG-12 — Organisation méthodique et alignement enforcé.** Une place mécanique pour chaque chose (une brique = un dossier à son nom) ; généré bat maintenu (vues générées depuis les registres, jamais éditées à la main) ; enforcé bat espéré (alignement fichiers-docs vérifié en CI, pas demandé).
- **REG-13 — Sécurité agentique : le contenu externe = données, jamais instructions.** Tout contenu non fiable (courriels, pages web, avis, pièces jointes, transcriptions) est traité comme donnée potentiellement hostile, jamais exécuté comme instruction. Défense en profondeur obligatoire : moindre privilège outillé, provenance marquée sur le contexte, actions à haut risque TOUJOURS derrière l'Approval Engine quelle que soit la source, filtrage entrées/sorties, tests adverses réguliers (rôle Grok 4.5), et mode ombre (l'agent planifie sans pouvoir exécuter) pour tout nouvel agent jusqu'à confiance établie.

2. Architecture macro — core autonome

Couche	Contenu	Notes
C1 — Infrastructure IA (hôte)	Noeuds, GPU, modèles LLM routés (passerelle), base vectorielle/RAG, stockage	Fournie par l'hôte ; exigences au README (section 3) ; branchée via Configuration
C2 — Core GRS + services transversaux	Moteur d'orchestration métier, files d'événements, planificateur (temps réel OU horaire), et les 8 services partagés : approbations, autorisation, audit/ledger, gates, notifications, mémoire (shim M10), registre de métriques, observabilité	Autonome, clé en main ; section 4
C3 — Modules M1-M10	Départements virtuels communiquant via le core ; contrats d'interface gelés	Un à la fois ; mocks derrière contrats tant que le module cible n'existe pas
C4 — Briques embarquées	Logiciels spécialisés livrés AVEC le core : vision, OCR, solveurs, MMM, analytique embarquée, marketing (41 briques M7)...	Candidates surclassables (REG-08)
C5 — Surfaces	Dashboard web (central), app mobile (4 profils), chatbot GRS, CRM tout-en-un	Sections 6 à 8
C6 — Données & mémoire	PostgreSQL (vérité structurée), snapshots Parquet (analytique), RAG multi-collections, ledger, sauvegardes	Séparation stricte dev / prod

Installation : branchement via le menu Configuration (noeuds, clés API, clé passerelle) ; aucun secret en dur. **Mode bridé :** sous le minimum, les départements temps réel basculent en horaire. **Numérotation :** les anciens chemins de dépôt (module6_equipes, module8_marketing...) sont périmés ; la M-numérotation et l'arborescence modules/m01..m10 font foi. **Déploiement clé-en-main (D-028) :** Docker Compose mono-noeud par défaut (simplicité opérationnelle, RAM rare) ; profil multi-noeuds documenté. **Profils d'installation (D-043, SIM-4) :** « infra-existante » (branchement sur passerelle/RAG/PostgreSQL fournis par l'hôte) OU « starter » (GRS auto-déploie passerelle, base vectorielle et PostgreSQL en conteneurs inclus — même le RAG devient clé en main ; l'utilisateur ne fournit que ses clés/endpoints de modèles). Wizard de premier démarrage : compte admin + MFA, français par défaut, MODE HORAIRE par défaut tant que le health-check n'a pas prouvé le temps réel (REG-11 opérationnalisé), chemin de désinstallation/export documenté.

3. README — Exigences de l'infrastructure hôte

3.1 Obligatoire (minimum vital)

- LLM en 3 tiers via passerelle de routage : local rapide (tri, classification), local fort (résumés, raisonnement, génération), premium (cas complexes, décisions majeures).
- Embeddings + base vectorielle (RAG multi-collections : mémoire, connaissances construction QC, fiscal, SEO/GEO, prix fournisseurs).
- PostgreSQL + stockage fichiers sécurisé (photos, documents, preuves). Capacité de planification (mode horaire complet).

3.2 Conditionnel — absent = mode bridé sur la fonction

- STT/TTS (M1 voix) — sinon texte seulement. Vision GPU détection + segmentation (M3) — sinon mesures manuelles à bornes larges. OCR (M5/M6) — sinon saisie assistée.
- Routage routier self-host ou API (M4) — sinon distance approximative. API météo (M4) — sinon vérification manuelle. Temps réel WebSocket + push (M4, mobile) — sinon horaire.
- Navigateur headless (crawl technique M7) ; GPU génération vidéo (M7 Wan 2.2) — sinon contenu image/texte seulement ; GPU pour MMM si Meridian retenu (PyMC-Marketing tourne sur CPU à l'échelle GRS).
- SMTP sortant avec SPF/DKIM/DMARC + réputation de domaine gérée (M7 courriel, M1/M5 notifications) — sinon envoi manuel.
- Adapters externes gated avec fallback souverain : téléphonie/SMS, courriel, comptable, banque, paiement, plateformes sociales/ads — CSV/manuel en repli.

- Règle : chaque manque a un fallback documenté ; GRS déclare son mode dégradé au dashboard, il ne simule jamais une capacité absente.

4. Services transversaux du core (ré-analyse croisée M1-M8)

Constat de la ré-analyse : les dix plans réinventent chacun les mêmes mécanismes. Sans unification, GRS maintiendrait six systèmes d'approbation, six pistes d'audit et cinq échelles de permissions divergentes. Décision D-010 : ces mécanismes deviennent des services du core (C2), consommés par tous les modules via des API de domaine. Chaque module garde ses règles métier ; le core fournit la mécanique.

Service	Ce que les plans réinventaient	Unification
Approval Engine	marketing_approval_request (M7, niveaux 0-6), FinancialApprovalRequest + StrategicApprovalRequest (M5, F0-F7 + GATED_CPA), seuils d'approbation M3, completed_pending_review M4, écarts matériaux M6, fusions doublons M2	Une file unique d'approbations au dashboard et au mobile, typée par domaine ; échelles mappées ; escalade, expiration, délégation et audit uniformes
Autorisation fine	F0-F7 finance, 0-6 marketing, rôles terrain, « le client voit SON chantier », « l'employé ne voit jamais les coûts »	Moteur ReBAC candidat : OpenFGA (CNCF, backends Postgres/SQLite, patterns agents IA/RAG/MCP) derrière un wrapper d'API de domaine ; sécurise aussi les agents au niveau ressource (moindre privilège)
Audit / Ledger	financial_audit_event (M5), audit stock (M6), accès GPS journalisés (M4), traçabilité règle #9 (M7)	Un service d'audit append-only du core, vues par domaine ; schéma ledger 10.5
Gate Engine	10 juges qualité M7, readiness photos M3, cohérence matériaux M6, honnêteté statistique M8	Un moteur de gates avec rulesets par domaine ; verdicts par dimension (REJ-0040), preuves attachées
Bus de notifications	M1 envoie, M4 notifie clients, M6/M8 alertent admin	Un bus unique : canaux (push, SMS, courriel), politiques opt-in, heures de silence, journalisation
Mémoire (shim M10)	Tous les modules écrivent des context packs M10	D-011/D-014 : contrat M10 RÉEL gelé en Phase 0 — event log immuable, context packs, registres (décisions, prompts, plans), Preflight/Postflight ; le shim implémente ce noyau, M10 complet le remplacera sans rien casser
Registre de métriques	Chaque module définit ses KPI ; M8 exige le dictionnaire canonique	Metrics-as-code : /registries/metrics/*.yaml versionné, PR-review, compilé vers l'entrepôt ; chaque module livre ses métriques À SON BUILD (M7 : CPL, coût/soumission, ROI par canal, part de citations IA)
Observabilité	Langfuse/OTel cités par M9 et M10 ; chaque module tracerait à sa façon	8e service : OpenTelemetry + conventions sémantiques GenAI (espace gen_ai.* : modèle, tokens, coûts, appels d'outils, raisons de fin) sur TOUTES les traces LLM/agents ; backend candidat : Langfuse self-host (endpoint OTLP natif) ; conventions encore expérimentales — versions épinglées (R-017)

- Sans D-011, chaque module mockerait la mémoire différemment — dette d'intégration garantie. Le shim coûte peu et gèle le contrat.

Fiabilité événementielle : outbox transactionnel + consommateurs idempotents (D-019). Le problème du double-écrit : mettre à jour la base ET publier un événement en deux opérations séparées = incohérence garantie au premier crash entre les deux. Solution standard : l'événement est écrit dans une table outbox DANS LA MÊME transaction que la donnée métier — l'événement existe si et seulement si le changement d'état a réussi ; un relais le publie ensuite (polling avec FOR UPDATE SKIP LOCKED, ou réplication logique). La livraison est au-moins-une-fois : les consommateurs sont donc OBLIGATOIREMENT idempotents (event_id unique vérifié en table inbox avant traitement) — sans cette vérification, on émet une facture en double, littéralement le scénario d'échec documenté. Ordre préservé par agrégat ; retries avec backoff ; dead letter queue ; le cas canonique GRS est la cascade de clôture M4 (six modules consommateurs : un crash à mi-cascade sans outbox = stock ajusté mais facture jamais créée).

Automatisations déterministes (D-027). Le planificateur du core est l'autorité. n8n reste une brique optionnelle pour des automatisations déterministes BORNÉES (règle des dix plans : jamais payer, supprimer, masquer un écart ou engager) — jamais sur le chemin critique, jamais en concurrence avec l'outbox.

5. Organisation des fichiers, docs et canon (REG-12)

5.1 Arborescence canonique

grs/ · **core/** (moteur, 8 services transversaux, safety guards) dont **core/frameworks/<Nom-Framework>/** pour les frameworks TRANSVERSAUX partagés par 2+ modules · **Modules M/** — LE dossier des départements virtuels : M1/, M2/, ... M10/, chacun clairement étiqueté (M1-front-office, M2-revenue-crm, ...) ; CHAQUE dossier MX/ contient : **frameworks/<Nom-Framework>/** (un dossier PAR framework propre au module, étiqueté au nom exact du framework), src/, tests/, contracts/ (pacts), metrics/ (déclarations canoniques), docs/, README.md · **surfaces/** (dashboard, mobile, crm) · **docs/** (index unique ; Diátaxis ; canon/ ; decisions/ ADR immuables) · **registries/** (frameworks.yaml, modules.yaml, metrics/, decisions.yaml, ledger/*.jsonl) · **ops/** · **.github/**

5.2 Règles

- Diátaxis pour la doc produit : chaque page appartient à UN quadrant (tutoriel, how-to, référence, explication) ; canon, ADR et ledger vivent À CÔTÉ, pas dedans. Aucun doc orphelin (tout atteignable depuis index.md) ; archives datées plutôt que suppression.
- Décisions = ADR : Proposed -> Accepted -> Deprecated/Superseded ; Accepted = immuable, changer = nouvel ADR qui supersède ; un ADR = une décision ; lié depuis le code.
- Historique horodaté : ledger JSONL append-only (preuves) + CHANGELOG généré depuis les Conventional Commits, jamais écrit à la main.
- Vues humaines (matrices, tableaux de statuts) GÉNÉRÉES depuis les registres YAML — jamais éditées à la main (leçon des matrices invalidées).
- **D-038 (RATIFIÉE NATHAN) — classification impeccable, méthodique, sans faille.** Les Modules M vivent dans LE dossier « Modules M », un sous-dossier par module (M1, M2, ...) clairement étiqueté. CHAQUE module a son dossier frameworks/. CHAQUE framework propre au module a SON dossier, étiqueté au nom exact du framework. EXCEPTION : les prérequis de l'infrastructure hôte (passerelle LLM, base vectorielle, PostgreSQL, embeddings, STT/TTS hôte...) ne vivent JAMAIS dans un module — ils sont la couche C1, listés au README-EXIGENCES. Rien de mélangé, rien d'éparpillé.
- Cas partagé (complément proposé, à ratifier) : un framework utilisé par 2+ modules vit UNE seule fois dans core/frameworks/ (même règle d'étiquetage) et chaque module consommateur le déclare dans son manifeste — la duplication par copie dans plusieurs modules est interdite : c'est la dérive de versions que REG-12 combat.
- Enforcement CI (mis à jour D-038) : (a) bijection registre-disque PAR MODULE — chaque entrée de frameworks.yaml existe sur disque avec FRAMEWORK.md + wrapper + tests AU BON EMPLACEMENT (Modules M/MX/frameworks/<Nom> ou core/frameworks/<Nom>), et chaque dossier de frameworks est au registre ; tout framework mal rangé = build rouge ; (b) ancres doc-code par empreinte (CI rouge si le code change après la doc) ; (c) front-matter obligatoire (owner, last_verified, covers) validé, seuils de fraîcheur ; (d) liens morts + style en lint.

6. Dashboard web — élément central

Menu Configuration (noeuds IP/user/mdp-clé SSH avec test de connexion, clés API modèles, clé passerelle, services, intégrations gated, mode temps réel/horaire par département) — tous les secrets via coffre-fort chiffré au repos (Vault / SOPS+age), masqués, rotation, accès journalisé ; rotation de la clé passerelle = CRITIQUE Phase 0. Vue d'ensemble (santé, activité modules, alertes, coûts LLM). **File d'approbations unifiée** (Approval Engine). Accès outils. Gouvernance (ledger, verdicts, audit). Administration (autorisation fine + MFA, sauvegardes testées).

7. « CRM » tout-en-un — support logiciel

CRM + comptable + inventaire en un, rôle de support MANUEL (GRS automatise déjà) ; base de données unique partagée, modifications journalisées et visibles des modules. Existant = actif : réutiliser-comme-brique vs réimplémenter décidé au consensus, Strangler Fig, tests de caractérisation obligatoires avant raccord.

8. Application mobile — 4 profils

Admin : inbox omnicanale, approbations unifiées, dispatch, alertes, rapports. **Vendeur :** leads + next-best-action, Client 360, soumission photos + géospatial sur place, coach. **Employé :** punch géorepéré (500 m + plages fixes), cédule/route, matériel à charger, clôture « Terminer les travaux » -> formulaire matériaux (M6), incidents, achats terrain reçu-photo, offline-first ; jamais coûts/marges. **Client :** profil auto à l'acceptation, dates + notifications, suivi « équipe en route » par lien tokenisé limité à SON chantier, factures + paiement, demande d'avis (consentement).

Architecture technique mobile (D-018)

Cadre candidat no 1 : React Native + Expo — TypeScript partagé avec le dashboard (une seule langue pour le rôle frontend), mises à jour OTA pour le clé-en-main ; alternative documentée : Flutter. **Synchronisation hors-ligne candidat no 1 : PowerSync (Open Edition self-host, gratuite : moteur + Sync Rules + tous les SDK)** — sync bidirectionnelle Postgres <-> SQLite local, base locale complète, file d'upload persistante ; les outils de service terrain et la collecte mobile sont précisément le cas d'usage nommé de sa voie d'écriture locale. Deux propriétés décisives pour GRS : (1) les LECTURES sont streamées par Sync Rules partitionnées par rôle/utilisateur — « l'employé ne voit jamais les coûts » s'applique donc à la couche de sync elle-même, pas seulement à l'UI ; (2) les ÉCRITURES passent par l'API GRS (server-authoritative) — chaque écriture traverse les gates du core, la validation et l'audit, exactement ce que l'anti-triche exige. Alternatives : ElectricSQL (sync en lecture, last-write-wins par défaut) ; WatermelonDB (contrôle total, mais sync maison à construire et maintenir). **Pas de CRDT** : la complexité CRDT ne se justifie que pour l'édition collaborative — la plupart des apps offline-first ont juste besoin d'écritures en file qui se synchronisent, un problème beaucoup plus simple ; c'est le cas de GRS.

Conflits : serveur-autoritaire ; conflit sur données sensibles (heures, matériaux, montants) = pending_review avec les deux versions présentées à l'admin — jamais d'écrasement silencieux (philosophie M4/M6). **Punch hors-ligne :** événement local signé (horodatage + fix GPS capturé AU MOMENT du tap + attestation d'appareil), validé côté serveur à la synchronisation, marqué OFFLINE pour l'audit ; anomalie = revue admin avec causes possibles, jamais d'accusation automatique (R-020).

Budgets de latence v1 (propositions internes, à ratifier au consensus) : réponse vocale M1 perçue < ~1 s ; API dashboard p95 < 300 ms ; synchronisation mobile complète < 30 s après reconnexion ; fenêtre des traitements batch 02:00-05:00. Ces cibles définissent ce que « temps réel » veut dire — sans elles, le mode bridé (REG-11) n'a pas de seuil mesurable.

Accessibilité des surfaces (D-033). Dashboard et app mobile : WCAG 2.2 niveau AA minimum, vérifié par les mêmes garde-fous automatisés que le site public (qui reste AAA — engagement Quadruple A de M7). Décision normative interne cohérente avec l'existant, à ratifier au consensus. Contraste, cibles tactiles, navigation clavier, lecteurs d'écran : un employé terrain avec des gants ou en plein soleil est un cas d'accessibilité, pas un cas marginal.

9. Carte des modules M1-M10

Règle d'interprétation (D-007) : plans = source du QUOI ; contexte d'implémentation antérieur périmé ; briques = candidates ; trous complétés au consensus.

Module	Département	Plan
M1	Front Office IA — réception omnicanale, voix progressive 0->5, qualification, adjointe administrative	Reçu
M2	Revenue CRM — Client 360, pipeline, scoring, relances gouvernées, anti-doublons, SLA, attribution	Reçu
M3	Estimation Intelligence — vision, mesures à bornes, prix/marge plancher, Monte Carlo, apprentissage	Reçu
M4	Field Operations OS — cédule/VRP, météo, GPS privé, punch, Work Completion Trigger	Reçu
M5	FinanceCore — 20 sous-départements, F0-F7, cash libre réel, forecast 5 ans, CFO Decision Report	Reçu (2 parties)
M6	Material Control OS — catalogue, PO, stock localisé, variance sans accusation, reorder	Reçu
M7	Marketing & Croissance IA — CMO IA, 41 briques, SEO/PPC/social, GEO/AEO, site Quadruple A	Reçu — PREMIER MODULE (D-008)
M8	Analytics — vérité chiffrée transversale, lecture seule, dictionnaire de métriques, XmR	Reçu + fondations arrêtées
M9	Strategic Intelligence OS — direction IA, War Room, Decision Engine, Risk & Opportunity Radar, 23 sous-modules	Reçu
M10	Mémoire Long Terme & Conscience Opérationnelle — non-oubli total, Forge Memory + GRS Business Memory	Reçu — contrat gelé (D-014)

Fiches M1-M6 (rappel condensé — inchangées depuis v0.2)

M1 : aucune opportunité perdue ; voix par phases jamais sautées ; interdits absolus (prix, dates, engagements, aveux) ; missed-call rescue opt-in ; digest quotidien ; EFVP majeure (enregistrements). **M2 :** aucun lead sans statut/owner/action ; readiness gate vers M3 ; fusion doublons = humain ; frontières prix=M3, dates=M4, finance=M5. **M3 :** anti-précision-inventée ; photos faibles = prix bloqué ; plancher rentable = alerte rouge ; approbation par seuils ; apprentissage estimé-vs-réel. **M4 :** readiness avant cédule ; scoring sans profilage ni sanction auto ; GPS heures de travail seulement, client = lien tokenisé ; Work Completion Trigger -> cascade M6/M5/M2/M7/M8/M10 ; écart = pending_review. **M5 :** solde différent d'argent disponible ; taux fiscaux versionnés sourcés sinon blocage ; fiscalité = RAG jamais fine-tunée ; argent réel F6 = Nathan+MFA ; fiscal F7 = Nathan+CPA ; adapters read-only -> dry-run -> write gated. **M6 :** price book sourcé sinon FALLBACK ; jamais « vol » automatique ; tout ajustement audité ; M6 ne paie jamais ; employé ne voit pas les coûts.

Fiche M7 — Marketing & Croissance IA (premier module)

- CMO IA, pas un agrégateur : la valeur vit dans le code custom (MarketingPlatformAdapter = coeur technique, Content Factory, Growth/Reputation/Ads/Competitive Engines, CMO Advisor M7.11) par-dessus 41 briques candidates qui font le travail ingrat.
- Adaptabilité sectorielle = intelligence du CMO Advisor qui pondère les trois piliers non négociables (SEO/PPC/social) selon l'entreprise cible — réutilisable pour d'autres projets solo de Nathan.
- D-009 : briques gratuites toujours ON ; payantes OFF par défaut sauf Local Falcon (visibilité IA géolocalisée = différenciateur no 1) ; chaque achat d'outil arbitré par ROI mesuré, gate humain niveau 4.
- Garde-fous : validation humaine 0-6 via l'Approval Engine ; backlinks jamais automatisés bout-en-bout (IA découvre/qualifie/rédige, humain valide/envoie, gate 3) ; WhatsApp self-hosted = violation ToS, exclu ; sourcing journalistique (Qwoted/Featured/SOS) = canal backlink AAA 100 % white-hat.
- Site web « Quadruple A » : Astro accessible + garde-fou Lighthouse/CWV à chaque génération + schema.org — Lighthouse 100 / WCAG AAA dès la première génération, pas en post-traitement.
- Workflows : chantier terminé -> contenu social (photos M4 -> Brand Core -> génération + vidéo -> gate -> publication -> mesure) ; rapport hebdo ; campagne saisonnière ; avis -> réputation ; surveillance AEO/GEO ; link building gated ;

sourcing journalistique.

- Loi 25 M7 : session replay (CRO) capture du PII -> fonction à consentement, jamais capture passive ; délivrabilité courriel = SPF/DKIM/DMARC + réputation par domaine (jamais mutualiser les domaines clients en mode SaaS).
- Agent Veille Marché : recherche en continu les techniques qui fonctionnent, alimente la base de connaissances — le département ne stagne jamais sur ses connaissances de départ.

Findings recherche intégrés à M7 (juillet 2026)

- **Postiz + Google Business Profile : CONFIRMÉ** (trou no 1 fermé). Intégration GBP documentée officiellement en self-host (OAuth Google, approbation d'accès API requise — quelques jours à une semaine), analytics GBP inclus (appels, itinéraires, clics). Caveat connu : bug multi-organisations (non applicable, une entreprise/une localisation). Fallback conservé : adapter API GBP direct. Bonus : Postiz Agent CLI + SKILL.md = pilotable nativement par Claude Code et les agents. AGPL-3.0 : Postiz reste un service séparé appelé par API (AGPL contenue, compatible vision SaaS).
- **GEO/AEO — nuance majeure par surface.** Position officielle Google (juillet 2026) : pour Google Search, llms.txt, « chunking » et mentions inauthentiques sont à ignorer — optimiser pour l'IA générative de Google = faire du SEO. MAIS les tactiques Princeton restent validées pour ChatGPT/Perplexity (citations, statistiques, citations d'experts : +30-40 % de visibilité), et la cohérence NAP multi-plateformes crée l'effet « moteur de consensus » (incohérence = l'IA omet l'info). Fondamentaux : crawlabilité des bots IA (Cloudflare les bloque maintenant PAR DÉFAUT — à vérifier), rendu côté serveur, requêtes fan-out. La base grs-seo-knowledge portera les DEUX positions + une tactique PAR SURFACE (Google AIO = SEO classique ; Perplexity = fraîcheur/autorité ; assistants = citations/consensus).
- **MMM — défaut ajusté : PyMC-Marketing à l'échelle GRS.** Benchmark quantitatif (PyMC Labs — biais vendeur à garder en tête) : 2-20x plus rapide, R^2 +19 %, erreur de contribution -40 %, et surtout 0,9-2,5 GB de RAM contre ~5 GB pour Meridian — tourne sur CPU au tier « startup, marché unique » qui est exactement GRS. Meridian (v1.7.0) reste l'alternative : GPU requis (testé T4 16 GB), Scenario Planner no-code (fév. 2026), données Google (volume de requêtes). Les deux = librairies de modélisation, pas des programmes de mesure — confirme l'avertissement du plan : le modèle = ~20 % du travail, la consolidation des données (10 ans CRM/compta) = le vrai chantier, propriété M8 ; M7.11 consomme les courbes de réponse.

Ordre interne de build M7 (vagues, dépendances gérées)

- Vague 1 — fondations sans dépendances : Brand Core, SEO technique (crawl), site Quadruple A, GBP + social via Postiz, base grs-seo-knowledge, registre de métriques M7. CRM existant = source lecture seule.
- Vague 2 — contenu + croissance : Content Factory, workflows chantier/saisonnier/AEO-GEO (photos de chantier historiques en attendant M4), Reputation, Competitive.
- Vague 3 — attribution/ROI minimal : Matomo + UTM + CRM (sans MMM). Ads Intelligence lecture + recommandation.
- Vague 4 — GATED derrière contrats gelés : MMM (arrive avec M8), leads entrants temps réel (M1), déclencheur chantier terminé et demande d'avis (M4). Mocks + statut GATED, jamais simulés comme fonctionnels.

Fiche M8 — Analytics (fondations arrêtées)

- Vérité chiffrée transversale : lit les vérités des autres modules par snapshots, ne possède aucune donnée source, n'écrit jamais dans les sources.
- Architecture 4 couches : snapshots Parquet append-only horodatés -> entrepôt embarqué DuckDB (in-process, MIT, zéro serveur, RAM-light, Parquet natif ; mono-écrivain = non-problème en batch read-only) -> couche sémantique -> surfaces. Rapports reproductibles : chaque rapport référence son snapshot_id (preuve au ledger).
- Dictionnaire de métriques = metrics-as-code (service transversal, section 4) : YAML versionné, PR-review avec impact chiffré des changements de définition ; MetricFlow/Cube = briques candidates si moteur externe voulu.
- Règle d'or : le LLM ne calcule JAMAIS un chiffre — la couche sémantique calcule déterministiquement, le LLM narre en citant metric_id + version + fraîcheur + n.
- Alertes = signaux XmR (process behavior charts) : limites naturelles calculées des données, règles de détection éliminant >99 % du bruit ; jamais de seuil arbitraire, jamais de conclusion accusatoire ; n affiché, « données insuffisantes » sous n minimal, comparaisons à périmètre saisonnier égal.
- Dashboards par rôle (l'employé ne voit jamais les marges — mécanique via l'autorisation fine) ; digests distribués par M1 ; mode horaire quasi intégral (batch nightly).

- Propriétaire du MMM (pipeline de données + modèle) ; M7.11 consomme.

Fiche M9 — Strategic Intelligence OS

- Cerveau de direction, pas un chatbot stratégique : répond à « que devrait faire l'entreprise maintenant, pourquoi, avec quelles preuves, risques, impacts, alternatives et validation ». Chaque sortie produit obligatoirement 15 éléments (état -> diagnostic -> preuves -> hypothèses -> incertitudes -> scénarios -> recommandations -> priorités -> impacts -> risques -> actions -> validation -> tâches par département -> suivi -> mémoire M10). Jamais de chiffre, revenu ou capacité inventés ; jamais d'action risquée sans approbation Nathan.
- Frontières nettes : M5 = CFO financier ; M8 = métriques ; M10 = mémoire ; M9 = interprétation, arbitrage, stratégie — au-dessus des départements sans les remplacer ni les court-circuiter.
- Chaîne : Strategic Data Fabric (snapshots M1-M8, fraîcheur vérifiée, détecteur de contradictions) -> Business State Graph -> semantic layer exécutif (CONSOMME le registre de métriques D-010, ne le réinvente pas) -> Council d'agents (CEO, COO, liaisons CFO/CMO, Sales, Ops, Risk, Data, Execution) orchestré par un Strategic Supervisor déterministe avec traces et scoring -> scénarios 7 j / 30 j / 90 j / 6 mois / 12 mois + stress tests -> Decision Intelligence Engine -> playbooks (croissance, crise, cashflow, saisonnier) -> Action Orchestrator (recommandation -> tâches assignées -> suivi de résultat).
- Rythme exécutif : War Room quotidien, rapport hebdo, revue mensuelle/trimestrielle, board memo ; dashboard exécutif + briefing mobile ; risque critique = notification Nathan + tâche + mémoire.
- Décisions via l'Approval Engine unifié ; stack alignée au squelette (LangGraph coeur, Temporal durable, Langfuse/OTel, évals Ragas/DeepEval) avec la règle « ne pas multiplier les frameworks ».
- Mode horaire quasi intégral ; position tardive dans l'ordre : un cerveau stratégique sans données réelles M1-M8 lirait du vide (même logique que R-009).

Fiche M10 — Mémoire Long Terme & Conscience Opérationnelle

- Principe absolu : rien n'est trop petit pour être mémorisé si cela peut influencer une décision future. M10 n'est pas un simple RAG : il capture, classe, relie, vérifie, révise, récupère et injecte — décisions, prompts, plans, fichiers, routes, tests, erreurs, raisons, instructions Nathan, conventions, état réel.
- Deux mondes reliés mais séparés par permissions et confidentialité : Forge Memory (l'usine : code, agents, décisions techniques, infrastructure) et GRS Business Memory (clients, procédures, prix, chantiers, apprentissages métier).
- Cinq règles absolues de non-oubli : (1) tout travail produit une trace complète (event, résumé, preuves, impact, prochaine action) ; (2) AUCUN agent ne code sans contexte — sinon STOP + alerte ; (3) rien n'est écrasé sans mémoire (historique, snapshot, rollback, event décision) ; (4) toute décision a une raison mémorisée (options, statut proposée/validée/remplacée/annulée) ; (5) la mémoire est révisable — fusion des doublons, vérités obsolètes marquées, jamais d'accumulation aveugle.
- Architecture 15 composants : event log immuable, structured memory, vector memory, knowledge graph, workflow checkpoints, prompt & plan registry, decision registry, code memory, business memory, self model, retrieval router, context injector, memory auditor, governance, evaluation harness.
- Workflows obligatoires : PREFLIGHT (16 étapes — constitution, matrice, décisions liées, erreurs historiques, dépendances, gates, contradictions, context pack, go/no-go) avant toute modification ; POSTFLIGHT (event, résumé de diff, snapshots, relations, sync vecteur/graph, TODO, signalement tests/docs manquants) après. D-014 : ces deux workflows deviennent LOI DU HARNAIS pour tous les agents dès la Phase 0.
- Audit no-forgetting périodique ; interdiction finale : jamais « M10 terminé » sans preflight, postflight, registres, context packs, sync vecteur/graph, tests E2E, docs et matrice à jour.

Fondations voix M1 (D-023)

- Cadre candidat no 1 : Pipecat — v1.0 (avril 2026), licence BSD-2 permissive, Python-first, pipeline de processeurs de trames où STT, LLM et TTS s'échangent en une ligne (60+ intégrations, aucun verrouillage), outillage de debug temps réel, et intégration OpenTelemetry native — alignement direct avec D-016.
- Pipecat Flows = machines à états conversationnelles structurées : c'est EXACTEMENT le mécanisme d'implémentation des phases voix 0->5 de M1 (chaque phase = un flow gelé, jamais sauté).
- Alternative documentée : LiveKit Agents — WebRTC natif + SIP intégré, latence moyenne mesurée 750-900 ms contre 800-950 ms pour Pipecat, un écart de 50-100 ms souvent négligeable ; à préférer si la téléphonie SIP directe

domine.

- Budgets de latence recalibrés sur données réelles : 800 ms bout-en-bout = le seuil où la conversation cesse de sembler robotique ; réalité de production : médianes industrielles 1,4-1,7 s et p99 à 3-5 s sur des millions d'appels ; la latence se mesure au p95, pas à la moyenne. Cibles GRS : <= 800 ms p50 perçu, <= 1,5 s p95 (réaliste) ; les phases 0-2 (voicemail, after-hours) tolèrent davantage.
- STT/TTS locaux (classes faster-whisper / Piper-Kokoro) = candidates Q1-Q6 ; fournisseurs cloud en repli gated ; VAD + barge-in obligatoires dès la phase 2.
- Limite 2026 assumée : les actions d'écriture où une hallucination coûte cher restent le point faible documenté des agents vocaux — la vraie défense de M1 reste ses interdits absolus + les gates, pas le modèle.

Fondations vision-mesure M3 (D-024)

- Hiérarchie de précision imposée : (1) RÉFÉRENCE D'ÉCHELLE explicite dans la photo — marqueur ArUco/ChARUCO fourni au vendeur, ou objet à dimension connue (porte, rangs de briques) = la seule voie vers une mesure de précision soumission ; (2) multi-vues / photogrammétrie quand plusieurs angles existent ; (3) profondeur métrique monoculaire = PRÉ-estimation à bornes larges, jamais un prix.
- Preuve chiffrée de la doctrine : sur un banc d'essai extérieur avec vérité terrain calibrée, le MEILLEUR modèle (Depth Anything V2) affiche une erreur absolue moyenne de 0,454 m (corrélation 0,962) et ZoeDepth dégrade à 3,09 m dehors — un demi-mètre d'erreur moyenne est incompatible avec un prix au pied linéaire. La borne large sans référence d'échelle n'est pas une posture prudente : c'est la physique du monoculaire.
- Briques candidates : Depth Pro (Apple, poids ouverts, 504M paramètres ~6 GB VRAM, carte de profondeur métrique 2,25 MP en 0,3 s GPU, SANS intrinsèques caméra, état de l'art zero-shot métrique + frontières) ; Depth Anything V2 ; Metric3D v2 — s'ajoutent à SAM2/Grounding DINO déjà au plan pour segmentation/détection.
- Le vrai levier de précision est dans l'app vendeur : guidage de capture (angles exigés, marqueur dans le cadre, distance recommandée) — pas dans le modèle. Photos non conformes = prix final bloqué (règle M3 existante, maintenant chiffrée).

Invariants transversaux (M1-M10)

Sources de vérité sans empiètement (M2 commercial, M3 prix, M4 terrain, M5 finance, M6 matériaux, M8 chiffres, M10 mémoire). Audit avant code ; statuts standardisés STUB / PARTIAL / GATED_NATHAN / GATED_CPA / GATED_EXTERNAL_API / NEEDS_HARDENING / DONE_VERIFIED (= routes + tests + docs + intégration + preuve). L'IA propose, source, score sa confiance — ne décide jamais la vérité finale. Validation humaine graduée sur argent, légal, fiscal, RH, client sensible, suppression, fusion. Anti-hallucination : jamais prix, date, garantie, précision ou chiffre inventés. Employés : jamais accusés, jamais profilés, jamais exposés aux coûts. Automatisations déterministes bornées (jamais payer, supprimer, masquer, engager).

10. Gouvernance multi-modèles

10.1 Roster initial — REMPLACÉ par l'équipe PROOF STACK (D-040)

- NOTE : la table ci-dessous est HISTORIQUE, conservée pour traçabilité. L'équipe officielle est celle de la méthode PROOF STACK (section 10bis, §2), ratifiée par Nathan.

Modèle	Rôle principal	Notes
Fable / Opus 4.8 (Claude Code)	Orchestrateur + validateur final	Mode délégation (coordonne, n'implémente pas)
GLM-5.2	Architecte + VÉRIFICATEUR PERMANENT no 1	Duo QA
DeepSeek V4 Pro	Codeur principal	Validateur de gate SAUF sur son propre code (CIV)
Qwen 3.7 Max	Codeur senior / revue croisée	Remplaçant CIV de DeepSeek
Composer 2.5	Frontend / dashboard / UI	xAI — R-001
Kimi Code 2.7	Intégration briques, refactor	
Grok 4.5	Red team / tests adverses	xAI — R-001
MiMo 2.5 Pro	VÉRIFICATEUR PERMANENT no 2 (contexte 1M)	Duo QA
Gemini 3.1 Pro	Multimodal, QA visuel UI	Suppléant duo QA
LongCat 2.0	Docs, génération massive de tests	
MiniMax M3	Données synthétiques, E2E, volume	

10.2 Processus

- Consensus de plan : pack compressé distribué à tous -> revues EN AVEUGLE, artefacts anonymisés -> verdicts par dimensions en choix forcé (REJ-0040) -> agrégation orchestrateur + débat ciblé -> consensus de tous + validation Fable/Opus + ratification Nathan -> plan gelé versionné.
- Quorum et dissidence (D-017) : l'objectif est l'unanimité. Si un blocage persiste après UN débat structuré ciblé, arbitrage Nathan avec dissidence DOCUMENTÉE au ledger (quel modèle, quelle dimension, quel argument). Un re-vote ne porte que sur les sections modifiées, jamais sur le plan entier — sinon boucle infinie.
- Gates : G0 conformité plan -> G1 exécution en zone bornée (worktree/branche) -> G2 duo QA en aveugle -> G3 triple validation (Fable/Opus + GLM + DeepSeek ou Qwen si CIV) -> G4 merge (CI verte, checks requis, preuves) -> G5 clôture (démo enregistrée + ledger + consensus).
- Anti-biais : auteur = pire réviseur de lui-même (exclusion) ; familles croisées ; obfuscation d'auteur ; ordre permuté ; verdicts par dimension ; panel majoritaire décorrélé si contesté.
- Ledger : entry_id, module, phase, tâche, auteur, vérificateurs, artefacts + hash, commandes, sorties, verdicts par dimension, verdict final, horodatage, commit/PR. Verdict masqué ou PHASE_ACCEPTED injecté = incident d'intégrité, phase invalidée.

10bis. Méthode d'exécution — PROOF STACK v1.1 (RATIFIÉE NATHAN, D-039)

Le TEMPLATE PROJET PROOF STACK COMPLET v1.0 (16 juillet 2026) est la méthode d'exécution officielle du plan, exécutée dans son intégralité avec Claude Code. Profil : **PROOF-Team** (3+ vendors disponibles ; repli Solo = rigueur réduite ANNONCÉE : 1 reviewer vendor différent du codeur + 3 passes séquentielles à stances distinctes en contextes séparés — jamais de fausse équipe). Règle transverse : chaque section et chaque étape se termine par PREUVE: <commande -> sortie> ou BLOCKED: <raison>. Rien d'autre. Aucune brique déclarée sans preuve d'activation.

Calibration (§0) — écrite au registre AVANT tout code

Identité projet + profil de rigueur (un seul, jamais mixé) au registre via ledger_append.py. Structuration : BMAD (défaut) + TDAD si argent/légal/sécurité/données critiques ; RÈGLE ABSOLUE : jamais deux méthodologies lourdes actives sur la MÊME story. Frontières T3 (humain seul) : paiements, secrets prod, suppressions définitives, publication/déploiement public, engagements externes. Rôle non pourvu par un vendor distinct = SIGNALÉ, jamais masqué.

Manifest des 12 briques de méthode (§1) — activation OBLIGATOIRE avec preuve

Couche PROOF (hooks git + session, skills, registres | commit-test : TODO rejeté, --no-verify bloqué) · Superpowers version PINNÉE (prompt test -> brainstorming) · LiteLLM proxy (Preuve A : routes réelles = 3 identités distinctes ; Preuve B : crew-* = passthrough) · Équipe multi-postes (CIV_MODELS : 3 réponses d'identité, 3 vendors) · mini-SWE-agent, exécuteur hors-session (1 story test -> .traj.json + PREUVE:) · Sentinelle tandem deepseek+glm (findings JSONL des DEUX modèles sur fichier stub test) · Canon Keeper tandem opus+gpt-5.5 (premier sweep -> rapport, divergences journalisées) · Canon A1-A5 (budget-check vert, ENTREE < 2K, gates doc verts) · DSPy + GEPA optimiseur OFFLINE (install + 1 dry-run ; adoption par PR SEULEMENT) · LLMLingua-2 compression docs/recherche SEULEMENT — jamais code/diff/pack de revue (ratio journalisé ou passthrough signalé) · Registres hash-chaînés (entrée DONE sans preuve rejetée par le gate) · eccAgentShield (scan -> zéro finding critique). Statuts admis : DONE-avec-preuve ou BLOCKED-avec-raison. Brique indisponible = BLOCKED journalisé, jamais une substitution silencieuse.

Équipe officielle (§2, D-040) — postes mappés sur mécanismes Claude Code réels

Poste	Modèle	Mécanisme / contraintes
Orchestrateur	Modèle actif (Fable / Opus 4.8)	Session ; signe SUR PIÈCES ; ne se relit JAMAIS — son code est relu par 3 vendors différents du sien
Architecte	GLM-5.2	Subagent crew-architect ; revue AVANT code ; ne code jamais
Codeur missions longues	Kimi K2.7 Code	crew-coder-long OU mini-SWE ; TDD strict
Codeur stories DIFFICILES	Kimi K3 (1M ctx)	mini-SWE ou session dédiée hors-session ; dispatch : story taguée complexité=haute ; TITULARISATION après firing test + tête-à-tête sur stories réelles ; aucun Kimi en revue de ses stories (même vendor)
Codeur bulk	DeepSeek V4 Pro	crew-coder-bulk OU mini-SWE
Codeur parallèle	Grok 4.5	mini-SWE, git worktrees ; jamais apparié à Composer en revue (xAI)
Reviewers aveugles x3	DeepSeek · Gemini 3.1 Pro · LongCat 2.0	civ_review.py HORS session ; 3 vendors distincts, tous différents du vendor du codeur ; SWAP CIV si reviewer = même vendor
Pool rotation	Kimi · Qwen 3.7 Max · MiMo 2.5 Pro	Réassignation journalisée, pilotée par les métriques
Sentinelle	Tandem DeepSeek + GLM	Daemon + hooks ; contrôle continu ; jamais codeuse ; peut ROUVRIR ; ne valide jamais
Canon Keeper	Tandem Opus 4.8 + gpt-5.5	Script ; cohérence du canon cross-story ; divergences journalisées

gpt-5.5 = vendor OpenAI ajouté au roster : décorrélation des familles accrue. Firing tests + titularisation + métriques de gouvernance (taux de rejet par modèle, bugs par poste, onboarding à froid) = la SUITE DE TESTS de la méthode elle-même ; les réassignations sont pilotées par les données, jamais par l'intuition.

Canon et anti-dérive (§3-4)

Canon A1-A5 instancié immédiatement ; test d'onboarding à froid périodique. 00-DIGEST.md (<= 400 tokens, UNE source) injecté par hook UserPromptSubmit tous les 10 tours ET sur événements — avant revue, avant toute déclaration DONE, après compaction (parité GUI vérifiée). Jamais la constitution complète en boucle : le contexte verbeux dégrade les agents et coûte. Chaque story démarre en contexte PROPRE (pack A1) ; stories découpées pour tenir dans un contexte ; la session longue de l'Orchestrateur est la seule exposée — d'où le digest. Doctrine : la mémoire améliore le chemin ; les GATES garantissent les sorties — un modèle qui a tout oublié ne franchit toujours pas un gate.

Gate de taille de story (D-044) — calibration METR

La durée de tâche est LE prédicteur no 1 d'échec des agents : près de 100 % de réussite sur les tâches qu'un humain finit en moins de 4 minutes, moins de 10 % au-delà de 4 heures ; et doubler la durée QUADRUPLE le taux d'échec (dégradation non linéaire dès ~35 minutes équivalent-humain). L'horizon de fiabilité à 80 % vaut environ le quart de l'horizon à 50 % — le meilleur modèle (février 2026) tient 14 h 30 à 50 %, donc la zone 80 % se situe vers 3-4 h. RÈGLE : chaque story porte une estimation en temps humain-équivalent ; CIBLE <= 1 h ; CAP DUR 3 h — au-delà, découpage OBLIGATOIRE avant assignation. Le dispatch « complexité=haute -> Kimi K3 » s'appuie sur cette estimation. Le taux d'échec par tranche de durée devient une métrique de gouvernance : le cap s'ajuste par les DONNÉES, pas par l'intuition. Validation croisée : en contexte long, des taux de 40-50 % s'effondrent sous 10 % même quand l'information reste présente — le contexte PROPRE par story (§4) n'est pas une préférence, c'est une nécessité mesurée.

Pipeline PAR STORY A->G (§5, D-041) — le seul chemin autorisé

- A — Préparation : A1 context pack scopé (slice canon + story + interfaces + objections/décisions passées par retrieval épisodique des registres ; préfixe stable pour le cache) ; A2 assignation codeur selon type (PAS l'Orchestrateur par défaut) ; A3 story structurelle -> revue Architecte AVANT code.
- B — TDD/TDAD : RED (tests d'abord, ils échouent) -> GREEN (passage RÉEL) -> REFACTOR ; Sentinelle continue pendant la génération.
- C — Gates locaux : lint · secrets · no-stub-scan (UN hit = bloqué, Orchestrateur inclus) · sweep des symboles touchés (code jamais ouvert affecté = retour B) · preuve au registre.
- D — PR + CI : JAMAIS de commit direct main ; tests unitaires + e2e + couverture + sécurité + vérification de hash + no-fake-done-check.
- E — REVUE AVEUGLE 3x3 : prompt GÉNÉRÉ depuis le template neutre (jamais écrit à la main, zéro indice d'auteur), 3 reviewers / 3 vendors, envoi SIMULTANÉ, verdicts SCELLÉS {verdict, objections[{sévérité, fichier, description, preuve}]}, révélés seulement quand TOUS sont déposés.
- F — Dépouillement PAR SCRIPT (jamais un LLM) : tous APPROVE + zéro objection critique. Sinon : Delphi anonymisé (max 2 rounds) -> arbitrage Orchestrateur SUR PIÈCES, journalisé.
- G — Merge : registre horodaté (qui, quoi, preuve, hash) · gate doc déclaratif (impact doc dans les critères ; doc:none journalisé sinon) · sweep Sentinelle post-merge.

Durcissements issus du walkthrough de la story pilote (SIM-2)

- C-09 — Bootstrap story 1 : le pack A1 de la toute première story se construit SANS retrieval épisodique (registres encore vides) ; journalisé comme tel, jamais simulé.
- C-10 — Le no-stub-scan porte une liste d'exclusions VERSIONNÉE (code généré par les frameworks, ex. squelettes d'endpoints) ; modifiable par PR seulement — sinon premier faux positif = premier contournement.
- C-11 — Le harnais e2e (Playwright + k6) s'installe en Phase 0 : l'étape D exige des e2e dès la story 1, pas « plus tard ».
- C-12 — Exception DOCUMENTÉE à D-032 : les verdicts de gate (APPROVE/REJECT) sont VOLONTAIREMENT sans échappatoire — le choix forcé est le mécanisme anti-biais ; l'incertitude s'exprime dans objections[], jamais dans le verdict. Aucun agent ne « corrige » ce schéma.
- C-13 — Story pilote définie : M7-S000 (Brand Core minimal — charger le profil de marque YAML + un endpoint GET + tests + doc), en MODE OMBRE, <= 1 h humain-équivalent. Elle prouve le pipeline, pas le produit.

- C-08 — Garde-fou hooks : un Stop-hook qui bloque inconditionnellement boucle à l'infini (piège documenté) — tout turn-guard porte une condition de sortie et un compteur.

Gate de FIN DE PHASE (§6, renforcement v1.1)

(1) e2e RÉEL de CHAQUE fonctionnalité livrée — le système FONCTIONNE, pas « les tests passent en isolation » ; (2) rapport de preuves (Synthétiseur) + rapport de santé (Sentinelle) + sweep Canon Keeper + test d'onboarding à froid si dû ; (3) REVUE AVEUGLE 3x3 DU RAPPORT DE PHASE — même mécanique scellée que l'étape E ; (4) signature de l'Orchestrateur SUR PIÈCES. Étape non validée = pipeline ARRÊTÉ, jamais contourné.

Registres, méthodologie-as-code et interdits (§7-8)

decisions.jsonl · preuves.jsonl · executions.jsonl · installations.jsonl — append-only, HASH-CHAÎNÉS, écrits via ledger_append.py, JAMAIS à la main : l'injection manuelle d'un verdict devient structurellement impossible, plus seulement interdite. Entrée DONE sans preuve = rejetée par le gate. Amendements de méthode = PR + circuit de revue, jamais de fork silencieux ; candidats GEPA = mêmes règles. INTERDITS (bloqués par les hooks) : stubs (pass, NotImplementedError, corps vides), TODO/FIXME/HACK dans un diff, tests sans assertion, assert True, skip non journalisé, données inventées, DONE partiel, affirmation factuelle sans source vérifiable, --no-verify, commit direct main, force-push main, auto-approbation, CLAUDE_CODE_SUBAGENT_MODEL global, compression de code/diffs/packs de revue. Mocks : dossier tests/ seulement, services externes seulement.

11. Méthodologie — BMAD v6 renforcée

- AMENDEMENT D-039 : structuration = BMAD (défaut) + TDAD (argent/légal/sécurité/données critiques) + Superpowers en couche workflow par story (version PINNÉE). La règle « jamais deux méthodologies LOURDES actives sur la MÊME story » remplace l'interdiction globale de coexistence de D-002.

BMAD v6 (planification agentique Analyst->PM->Architect => PRD + Architecture ; développement contextuel SM->Dev->QA par stories autoportantes ; docs = source de vérité dans Git ; dev = « artisan obéissant » qui S'ARRÊTE ET RAPPORTE en cas de conflit ; module Test Architect, QA vision pour l'UI). Rôles mappés au roster ; coût neutralisé par routage local. Jamais BMAD + autres cadres superposés sur un même module ; argent/légal = cycle complet. Claude Code Agent Teams (expérimental) : lead en délégation, coéquipiers à contexte propre, 5-6 tâches autoportantes, un worktree Git par agent, zones d'écriture bornées. TDD RED-GREEN-REFACTOR ; caractérisation obligatoire avant raccord de l'existant ; bogue trouvé = test de non-régression avant correction. DoD : tests + lint + scans + docs + démo + ledger + sous-matrice à jour.

Contrats gelés = contrats EXÉCUTABLES (D-015). Chaque contrat d'interface inter-modules est formalisé en contrat dirigé par le consommateur (Pact) : le consommateur enregistre ses attentes sous forme de tests, le fournisseur les vérifie dans son propre pipeline — sans que les deux services tournent ensemble, à la vitesse des tests unitaires. Les pacts sont des artefacts versionnés (broker) et le gate « can-i-deploy » s'intègre à G4. Le mock GATED d'un module absent devient ainsi vérifiable : quand le vrai module arrive, il doit satisfaire le pact, pas une intention. Justifié ici car chaque contrat GRS a PLUSIEURS consommateurs (le trigger de clôture M4 est consommé par M6, M5, M2, M7, M8, M10 ; le shim M10 par tous).

Migrations de schéma : expand-contract obligatoire (D-020). Règle : ne jamais changer le schéma et le code qui en dépend dans le même pas — c'est la cause racine des pannes de migration. Tout changement cassant se découpe en trois déploiements : EXPAND (ajout additif, rien de retiré), MIGRATE (double-écriture ancien+nouveau puis backfill par lots bornés — jamais un UPDATE géant qui verrouille la table), CONTRACT (retrait de l'ancien seulement quand plus rien ne le lit — seule étape irréversible : aucun prix à se presser, l'ancienne colonne ne coûte presque rien, la retirer un jour trop tôt coûte cher). Les migrations sont du code de premier ordre : revues, testées sur des données de taille production (une opération en millisecondes sur 1 000 lignes peut verrouiller des minutes sur des millions), liées aux releases. Outil candidat : Alembic (core Python) ; changement de schéma = point d'approbation distinct (comme les dépendances). Clé-en-main : chaque release GRS embarque son bundle de migrations + chemin de retour documenté.

Standard d'API des surfaces (D-025). OpenAPI = source des contrats de surface (Pact reste le mécanisme inter-modules). Erreurs : RFC 9457 Problem Details — standard IETF (Standards Track, remplace le 7807) : cinq champs machine-lisibles, fini les formats d'erreur maison. Idempotency-Key obligatoire sur tout POST non sûr — un brouillon IETF Standards Track formalise l'en-tête popularisé par Stripe ; c'est le prolongement de D-019 jusqu'au bord externe (mobile hors-ligne qui rejoue ses écritures). Retries : backoff exponentiel + jitter + plafond — la mathématique

d'Amazon est brutale : trois retries empilés sur trois couches transforment UNE requête en 243x de charge sur la base en aval. Versionnage /v1 ; pagination par curseur. Tests : Playwright (E2E des surfaces) et k6 (charge — valide les budgets de latence de D-018/D-023) = candidates au pipeline Q1-Q6.

Gates d'évaluation LLM en CI (D-030). Golden datasets versionnés : chaque cas capture entrée + contexte + résultat attendu, sourcé des échecs réels, re-jouable entre versions de modèles et d'app. À CHAQUE PR touchant un prompt, un agent, une collection RAG ou un routage de modèle : les métriques tournent sur les goldens, comparées à la baseline stockée — régression au-delà de la tolérance = build rouge. Les scores qui échouent cassent le build : les régressions n'atteignent jamais la production. Pattern deux-outils confirmé par l'industrie : cadre léger de gating CI (DeepEval style pytest, Ragas pour le retrieval, Promptfoo pour la comparaison inter-modèles et le red teaming) + plateforme de suivi et d'annotation (Langfuse, déjà D-016). Promptfoo = l'outil du SURCLASSEMENT DE MODÈLES : REG-08 s'étend aux modèles — valider qu'un modèle moins cher ne régresse pas sur les goldens AVANT de basculer. Mapping métrique -> hyperparamètre : pertinence contextuelle -> taille de chunk / top-K / embedding ; fidélité -> choix de LLM. Tout changement de prompt système = PR avec run d'évaluation attaché (preuve au ledger).

Sorties structurées : trois couches + règle anti-fabrication (D-032). Standard : (1) schémas typés (Pydantic/zod) ; (2) contraintes côté génération (décodage contraint par grammaire/FSM — moteurs type Outlines/XGrammar ou modes natifs des fournisseurs) ; (3) validation sémantique post-parse. Les prompts sont un renfort auxiliaire, JAMAIS la garantie structurelle primaire. RÈGLE D'OR : l'enum sans échappatoire est le mécanisme par lequel un schéma rigide FABRIQUE de l'hallucination — le modèle, privé du droit d'être incertain, choisit la mauvaise catégorie la plus proche avec une confiance structurelle totale. Donc : chaque champ catégoriel porte une valeur d'échappement explicite (unknown / other / not_present) et un champ légitimement absent est nullable, jamais required. Mécanisme prouvé : masquer les tokens à haute probabilité distord la distribution — sortie syntaxiquement valide mais sémantiquement dégradée (NeurIPS 2024) ; sur les petits modèles, le décodage contraint peut créer des pièges de format et des boucles (« structure snowballing »). Composé gagnant documenté : ANCRER les valeurs (RAG) puis CONTRAINDRE le format — les deux se cumulent. S'applique partout : extractions OCR M5/M6, verdicts de gates, sorties d'agents, réponses M8.

12. GitHub à plein potentiel

main protégée ; branches agent/<ticket>-<desc> ; PR obligatoires ~500 lignes max, checks requis (tests, lint, gitleaks, semgrep/SCA, build, lint de structure REG-12, ancres doc-code, validateur front-matter), approbations périmées rejetées à chaque commit, CODEOWNERS ; trailer Co-Authored-By par modèle ; dépendances = approbation distincte ; CI rapporte QUELS gates ont tourné ; Issues/Projects = plan visible ; milestones par module ; releases taguées ; ADR pour toute décision ; re-approbation forcée si un agent change les hypothèses d'un autre.

Chaîne d'approvisionnement du build agentique (D-035). Contexte 2026 documenté : vague d'attaques sur les écosystèmes — Trivy, axios, TanStack (84 versions malicieuses publiées en six minutes, portant une provenance SLSA Build L3 VALIDE), détournements de tags d'actions — et **LiteLLM, notre passerelle, compromis sur PyPI en mars 2026**. Règles obligatoires : (1) toute Action GitHub épinglée au SHA de commit complet — les tags sont mutables, seul le SHA est immuable ; l'incident Trivy aurait été évité par l'épinglage ; (2) lockfiles commités + installations gelées : npm ci échoue sur toute divergence et ne réécrit jamais le lockfile ; pip avec vérification de hachages ; (3) scripts de cycle de vie désactivés par défaut (ignore-scripts) avec liste blanche explicite — caveat documenté : certains vecteurs contournent ce drapeau, d'où la défense en profondeur ; (4) délai de quarantaine sur les nouvelles versions (cooldown / minimumReleaseAge) — devenu défaut d'écosystème chez pnpm, npm, Homebrew et VS Code après la vague ; (5) GITHUB_TOKEN à moindre privilège, CODEOWNERS obligatoire sur tout changement de workflow, jamais pull_request_target pour du code de fork ; (6) SBOM + provenance Sigstore/SLSA en sachant qu'une signature valide prouve L'ORIGINE d'un artefact, pas son innocuité ; (7) briques et poids de modèles : SHA-256 épinglé au registre BRK et vérifié à chaque installation ; (8) les scanners eux-mêmes sont des cibles (Trivy compromis) : épinglés, à jour et isolés des secrets de production.

Cartographie GitHub de D-038 — exploitation à 100 %. L'arborescence EST le contrat du dépôt : CODEOWNERS par module (Modules M/M7/ -> propriétaires et validateurs du module), labels module:mX automatiques par chemin, milestones et releases PAR MODULE (grs-m7-v1.0.0), branches agent/mX-<ticket>, gabarit de PR avec case obligatoire « arborescence D-038 respectée », et le lint de structure (5.2) en CHECK REQUIS de branche — un framework mal rangé ne peut physiquement pas merger. Projects = plan visible par module ; Issues liées aux stories BMAD ; environnements par surface. Le dépôt et le plan ne peuvent pas diverger : la CI l'interdit.

13. Économie de tokens et mémoire

Plus petit ensemble de tokens à haut signal. Préfixe stable en tête (cache ~10x moins cher), variable en queue ; compaction à 60-70 % ; purge des sorties périmées ; références par ID (BRK-0042, M7-S03) plutôt que recopie ; artefacts sur disque/Git (le contexte reçoit chemin + hash) ; grosses lectures déléguées à des sous-agents. Mémoire longue : RAG prod séparé du dev ; MCP mémoire/M10 = index, la vérité reste Git + ledger ; context packs par module via le shim M10 ; note de passation compressée en fin de session.

Ingénierie de récupération par collection (D-029). Pipeline standard imposé : réécriture de requête (multi-tours) -> récupération HYBRIDE (dense top-50 + BM25 top-50) fusionnée par Reciprocal Rank Fusion -> reranking cross-encoder sur la shortlist -> top 5-8 -> contexte avec métadonnées et citations obligatoires. Preuve : sur 23 088 requêtes de documents FINANCIERS (texte + tableaux), hybride + reranking atteint Recall@5 de 0,816 contre 0,695 pour l'hybride seul (+17,4 %) et 0,587 pour le dense seul (+39 %), toutes différences statistiquement significatives — et sur ces documents financiers, BM25 bat le dense sur presque toutes les métriques (codes, numéros, termes exacts) : directement applicable au RAG fiscal M5 et au price book M6. Nuance honnête à encoder : sur certaines tâches, ajouter BM25 aide les petits modèles d'embedding mais DÉGRADE les meilleurs — on mesure par collection (D-030), on n'assume jamais. Chunking structure-aware (titres/sections) + chevauchement modeste ; reranker sur shortlist (top-100 = millisecondes), jamais sur le corpus entier. Changement de modèle d'embedding = migration d'index complète, versionnée, évaluée avant bascule (R-026).

14. Sécurité et conformité Loi 25

14.1 Technique

Exposition minimale (un seul noeud public, TLS, pare-feu) ; coffre-fort secrets ; autorisation fine centralisée (le contrôle d'accès cassé est le risque no 1 de l'OWASP — une seule implémentation à auditer plutôt que huit) ; MFA ; sauvegardes ET restaurations testées ; journaux inviolables ; scans continus (SCA, semgrep, gitleaks) ; patrons dangereux bloqués en CI ; séparation dev/prod stricte ; données synthétiques en test ; webhooks : signature, idempotence, anti-replay, rate limiting, dead letter queue.

Sécurité agentique (REG-13). L'injection de prompt est le risque no 1 de l'OWASP LLM Top 10 et ne se corrige pas par un correctif : elle exploite la conception même des LLM ; ni RAG ni fine-tuning ne l'éliminent — la mitigation est la défense en profondeur (moindre privilège outillé, filtrage entrées/sorties, approbation humaine des actions sensibles, tests adverses réguliers). Surfaces GRS les plus exposées : M1 (lit des courriels et voicemails externes), M7 (crawle le web), M6 (courriels fournisseurs) — l'injection INDIRECTE y est l'attaque de référence, avec précédent réel en production (EchoLeak, exploitation zéro-clic). Politiques core : provenance marquée sur tout contenu externe ; aucun outil à effet de bord appelable depuis un contexte souillé sans gate ; secrets JAMAIS dans le contexte des agents ; journalisation exacte prompt/sortie/outil/raison ; mode ombre pour tout nouvel agent.

Coupe-circuits de coût LLM. Les charges agentiques tournent en boucles : sans plafond, un emballement est un désastre financier. Budgets par clé, par agent et par module au niveau de la passerelle, alertes à 70 %, coupe-circuit dur à 100 % avec escalade humaine ; kill-switch global au dashboard. Capacité de budget par clé à confirmer sur la passerelle lors de la rotation Phase 0 (P0-4).

Sauvegarde et reprise (D-021). pgBackRest : complet hebdomadaire + différentiel quotidien + archivage WAL continu => restauration à la seconde près (PITR) — le seul mécanisme qui récupère une suppression accidentelle ou une corruption applicative sans perdre les heures depuis le dernier dump. Dépôt HORS HÔTE et chiffré (aes-256-cbc) : une sauvegarde sur le même disque est détruite par la même panne. Règle 3-2-1. Cibles : RPO <= 15 min (WAL) ; le RTO n'est PAS un chiffre de documentation — c'est la durée mesurée du dernier exercice de restauration. Doctrine : une sauvegarde jamais testée n'est pas une sauvegarde — exercice de restauration AUTOMATISÉ hebdomadaire avec preuve au ledger ; un test de restauration raté = incident de sévérité 1 (le plan de reprise est cassé avant que le désastre ne le prouve) ; alerte si la dernière sauvegarde réussie a plus de 24 h. Nuance PITR : le rejeu WAL est glouton par défaut — pour la corruption, viser un point dans le temps (--type=time) ou l'état cohérent immédiat, sinon l'erreur est rejouée. Couverts séparément : snapshots Parquet, registres (dans Git), coffre-fort de secrets.

Hygiène des tables à forte croissance (D-034). Outbox, inbox d'idempotence, événements, ledger et journaux d'audit croissent sans borne par conception : partitionnement temporel dès la création, archivage des lignes publiées/consommées vers le stockage froid (Parquet), rétention par classe alignée sur 14.2, et pooling de connexions (PgBouncer candidat) devant PostgreSQL. Pratique standard, à ratifier au consensus ; sans elle, la table outbox devient le goulot du système en quelques mois.

Sandbox d'exécution du code agent (D-036). Consensus explicite des praticiens en 2026 : l'isolation Docker/runc à noyau partagé ne suffit PAS pour du code généré par IA — un exploit noyau dans le conteneur peut s'échapper vers l'hôte. Étages GRS : code non fiable, code généré par agent et briques en essai Q4 = micro-VM (Firecracker : démarrage ~125 ms, moins de 5 MiB par VM, snapshot-restore en 5-30 ms pour les sessions multi-tours — le noyau dédié bloque des classes entières d'évasion) ou Kata Containers ; gVisor = intermédiaire (noyau espace-utilisateur, mais bloque le passthrough GPU — Firecracker/VFIO le permet si la brique a besoin du GPU) ; conteneurs durcis (seccomp, AppArmor, capabilities réduites, système de fichiers en lecture seule, réseau sortant filtré par liste, aucun socket Docker exposé, limites CPU/RAM/temps) réservés au code REVU et de confiance. Secrets jamais montés dans une sandbox. Ordre de grandeur rapporté (source unique, indicatif) : ~90 % d'incidents en moins vs agents à accès hôte libre. Deux couches complémentaires et toutes deux requises : l'isolation d'exécution (ici) et les menaces de couche agent — injection, empoisonnement d'outils (REG-13).

Réponse à incident (D-037). Runbook versionné au canon : isoler (couper l'accès, geler les déploiements) -> révoquer et faire tourner les secrets touchés -> restaurer par la chaîne prouvée (P0-9) -> notifier selon 14.2 (registre d'incidents, CAI si préjudice sérieux, personnes concernées) -> post-mortem SANS BLÂME au ledger avec actions correctives suivies jusqu'à clôture. Exercice annuel sur scénario (ex. : clé de passerelle compromise). Un incident sans post-mortem tracé = le même incident deux fois.

Identité et authentification (D-022). IdP self-host candidat no 1 : Zitadel — Go, binaire unique + Postgres, ~100 MB de RAM (contre ~300 MB pour Authentik en multi-services Python, et Keycloak/Java reconnu pour son poids opérationnel et son tuning manuel) ; architecture event-sourcée = journal d'audit immuable natif des opérations d'identité (aligné à la doctrine de preuve) ; organisations multi-tenant natives (l'avenir SaaS est déjà dans le modèle) ; OIDC/OAuth2/SAML, WebAuthn/FIDO2 passwordless (employés), MFA (admin), magic links / passkeys (portail client). Caveats documentés : AGPL 3.0 depuis 2025 => service séparé appelé par API (AGPL contenue, même règle que Postiz/Metabase) ; pas d'interface LDAP (non requis) ; un IdP n'est PAS la couche d'autorisation — il s'appaire au moteur de permissions ressource-niveau (OpenFGA, D-010), exactement l'appairage recommandé. Alternatives documentées : Authentik (écosystème d'intégrations le plus large), Keycloak (CNCF, mature).

Modélisation des menaces par module (D-026). Cadre des quatre questions (Threat Modeling Manifesto / Shostack) au moment du PRD+Architecture BMAD de chaque module : que construit-on ; qu'est-ce qui peut mal tourner ; qu'est-ce qu'on fait ; a-t-on bien fait. STRIDE structure la deuxième question — la méthode la plus mature, choisie pour son faible taux de faux positifs — et son jumeau vie privée LINDDUN s'applique aux surfaces Loi 25 (GPS, enregistrements, session replay, punch). Garde-fou de Shostack intégré : une definition-of-done légère, sinon les équipes gament l'exercice et la valeur disparaît. Sortie : registre de menaces au ledger, mitigations liées aux stories, et Grok 4.5 (red team) rejoue le modèle de menaces en G2 avec des tests adverses.

14.2 Loi 25 — intégrée au produit

- EFVP obligatoire pour tout développement de système impliquant des RP (GRS entier) et avant communication hors Québec (modèles cloud) — auto-hébergement = avantage : RP locales par défaut, cloud sur données minimisées.
- Technologies d'identification/localisation/profilage (punch, GPS, tracking client, enregistrements M1, session replay M7) : information préalable, politique écrite, consentement documenté, collecte limitée aux heures de travail, rétention définie, accès journalisés.
- Minimisation, confidentialité par défaut, calendrier de destruction, droits outillés (accès, rectification, retrait, portabilité), registre d'incidents 5 ans + notification CAI si préjudice sérieux, responsable RP = Nathan.
- Sanctions jusqu'à 25 M$ / 4 % du CA mondial ; mode SaaS futur = obligations rehaussées + réputation d'envoi courriel par domaine client (jamais mutualisée).
- Rétention par classe de données : les registres et pièces justificatives fiscales (factures, reçus, journaux, inventaire, paie, TPS/TVQ) sont conservés SIX ANS après la dernière année visée — sur le même support et sous forme intelligible s'ils sont électroniques, intégrité maintenue ; plus longtemps en cas d'opposition ou de contestation ; destruction anticipée seulement sur autorisation écrite du fisc. Le calendrier de destruction Loi 25 s'applique aux AUTRES classes (enregistrements d'appels, traces GPS, session replay, photos non contractuelles) avec des durées courtes définies par politique — la conservation fiscale est l'exception légale documentée à la minimisation.
- Langue (Loi 96 / Charte de la langue française) : les sites web, logiciels et applications offerts aux clients québécois doivent être accessibles en français dans des conditions équivalentes à toute autre langue ; contrats de consommation et d'adhésion en français sauf demande expresse documentée ; documents de travail des employés et service à la clientèle en français. GRS = FRANÇAIS D'ABORD sur toutes les surfaces (portail client, soumissions, factures, notifications, app employé, site Quadruple A). En mode SaaS : argument produit direct — sanctions de 700 $ à 90 000 $, francisation OQLF dès 25 employés depuis juin 2025, et la loi s'applique même aux entreprises hors Québec qui servent des clients québécois.
- Rédaction PII avant le cloud (D-031) : couche de dé-identification OBLIGATOIRE sur tout flux vers un modèle cloud, les journaux et les traces. Brique candidate : Presidio (open-source, NER + regex + checksums ; analyzer + anonymizer ; module image-redactor pour photos/documents scannés ; module structured pour les tableaux ; opérateurs réversibles chiffrés). Pipeline complet : entrée -> analyse/anonymisation -> récupération RAG -> rédaction DES CHUNKS récupérés (piège documenté : les documents récupérés contiennent souvent plus de PII que le message lui-même) -> prompt -> LLM -> rédaction de la sortie -> trace rédigée. Doctrine : traiter le modèle cloud comme un processeur tiers NON FIABLE — assainir à l'entrée, reconstruire à la sortie via mapping réversible sous clé locale. Le pattern local-filtre-avant-API-distante réduit la fuite de PII de 100 % à 7,5 % en conservant la qualité de réponse — notre architecture à tiers locaux le permet nativement. Limites assumées : détection au niveau message, sans état inter-sessions ; cohérence des remplacements = mapping maison versionné. C'est l'opérationnalisation concrète de « cloud sur données minimisées » (EFVP).

15. Stratégie briques — embarquées et surclassables

Terminologie unifiée (D-038) : « framework » et « brique » désignent la même chose ; la fiche d'identité devient FRAMEWORK.md ; les identifiants de registre restent BRK-xxxx. Pipeline Q1-Q6 : licence (MIT/Apache/BSD OK ; GPL/AGPL = évaluation — les briques AGPL comme Postiz ou Metabase restent des services séparés appelés par API, AGPL contenue) ; vitalité ; sécurité (scan + revue scripts d'installation) ; essai sandbox + benchmark vs candidate en place ; wrapper d'interface + tests de caractérisation (remplacement sans casser GRS) ; fiche ledger BRK-xxxx (version épinglée, hash, adaptations, surclassements). Brique modifiée = fork interne versionné ; mises à jour amont revalidées ; surclassement = Q1-Q6 complet + bascule par wrapper (REG-08).

16. Feuille de route

Phase 0 — Fondations (avant M7)

- P0-1 : Dépôt GitHub selon REG-12 : arborescence canonique, constitution (canon/), CI complète incluant les gates d'alignement.
- P0-2 : BMAD v6 + Claude Code + Agent Teams ; définitions d'agents ; passerelle branchée.
- P0-3 : Ledger + registres initialisés (bricks, modules, metrics, decisions) + vues générées au dashboard.
- P0-4 : Coffre-fort secrets + ROTATION DE LA CLÉ PASSERELLE (critique ouvert).
- P0-5 : Squelette dashboard : MFA + autorisation fine (moteur candidat OpenFGA derrière wrapper), Configuration branchée au coffre, page santé, indicateur de mode.
- P0-6 : Services transversaux v0 : Approval Engine, bus de notifications, Gate Engine, audit, SHIM M10 (contrat gelé D-011), registre de métriques.
- P0-7 : Gabarit EFVP + politiques (consentement SMS/enregistrements/géoloc/session replay) + registre d'incidents.
- P0-8 : Consensus : distribution du PACK_CONSENSUS_GRS_v1 aux 11 modèles -> protocole 10.2 (revues en aveugle, verdicts par dimension) -> agrégation -> ratification Nathan (rôles, ordre des modules, trous M7, briques) -> GO M7 vague 1.
- P0-9 : Chaîne de sauvegarde pgBackRest opérationnelle (WAL + dépôt hors hôte chiffré) + PREMIER exercice de restauration chronométré et prouvé au ledger (cette durée = le RTO réel) ; IdP déployé, MFA admin actif.
- P0-10 : CHECKLIST PROOF STACK (§9) — calibration au registre ; 12 briques du manifest DONE-avec-preuve ou BLOCKED journalisé ; équipe mappée + firing tests au registre ; CANON/ instancié, gates doc verts, budget vert, test à froid no 1 ; digest + hook anti-dérive actifs, injection vérifiée ; BMAD brief -> PRD -> architecture -> stories ; PREMIÈRE STORY PILOTE de bout en bout (pack -> codeur -> TDD -> gates -> PR -> 3x3 scellée -> tally -> merge -> registre) = preuve vivante que le pipeline fonctionne AVANT la story 1 de M7.

Ordonnancement prouvé (SIM-3, C-14). Une PHASE C (consensus) précède la Phase 0 : le consensus est OUTILLÉ — consensus_run.py (brique de méthode no 13) distribue le pack aux 12 modèles via les routes de la passerelle, collecte les verdicts JSON scellés et les dépouille par le tally ; jamais de copier-coller manuel entre 12 interfaces. Ordre Phase 0 corrigé par l'analyse de dépendances : repo (P0-1) -> coffre + ROTATION de la clé (P0-4 AVANT la passerelle) -> routes + preuves A/B + LiteLLM épinglé par hash (P0-2) -> ledger/registres/hooks (P0-3, P0-6) -> scripts de méthode TESTÉS avec leur propre suite (ledger_append, civ_review, tally — R-030) -> harnais e2e (C-11) -> dashboard shell (P0-5) -> EFVP gabarit (P0-7) -> sauvegardes + IdP (P0-9) -> checklist §9 + story pilote M7-S000 (P0-10). MARQUAGE : items BLOQUANTS-STORY-1 = repo, clé, routes, ledger/hooks, scripts testés, harnais e2e, pilote ; PARALLÉLISABLES pendant M7 vague 1 = dashboard complet, EFVP par module, IdP complet ; la chaîne de sauvegarde devient BLOQUANTE avant la première DONNÉE DE PRODUCTION, pas avant la story 1 (Git couvre le code). Zéro dépendance circulaire détectée.

Goulot humain (SIM-1, C-04) : les validations T3 et approbations s'empilent sur UNE personne — budget de validation quotidien (créneau fixe), file priorisée par impact, digest plutôt qu'interruptions, SLA interne d'approbation journalisé. **Environnement canonique (C-07) :** Claude Code s'exécute sur Ubuntu/WSL2 (hooks bash) — jamais un pipeline moitié Windows natif, moitié Linux. **Anti-échappatoire doc (C-06) :** le taux de doc:none par module est une métrique de gouvernance avec seuil d'alerte — sinon doc:none devient le défaut silencieux.

Critère de sortie : chaque item avec preuve au ledger. Aucune exception.

Phase 1 — M7 par vagues (puis un module à la fois)

Cycle : plan gelé -> BMAD planification (PRD + Architecture + contrats d'interface gelés vers M1/M4/M8/M10) -> stories -> exécution multi-agents G0-G5 par vague (ordre interne section 9) -> démonstration 100 % fonctionnelle -> re-tests ->

clôture consensuelle -> module suivant (ordre à ratifier ; signal des plans : la boucle terrain/données réelles ensuite, M5 quand les données existent).

17. Checklist maîtresse

Exigence	Où
Core autonome clé en main ; README d'exigences ; mode bridé horaire	REG-11 ; 2, 3
Briques intégrées au core, jamais figées, surclassement automatique	REG-08 ; 15
Organisation méthodique : une brique = un dossier, docs/canon sans éparpillement, registres horodatés, alignement fichiers-docs enforcé	REG-12 ; 5
Services transversaux unifiés (approbations, autorisation, audit, gates, notifications, mémoire, métriques)	4 (D-010)
10 modules M, un à la fois, 100 % prouvé ; M7 premier (D-008) avec ordre interne par vagues	REG-06 ; 9, 16
Consensus tous modèles + validation Fable/Opus ; étapes validées trio avec rotation CIV ; duo QA indépendant en aveugle	10
Faux done/stubs interdits ; preuves au ledger ; statuts standardisés	REG-01/02 ; 9, 10.2
GitHub plein potentiel ; BMAD ; économie de tokens ; RAG + MCP mémoire	12 ; 11 ; 13
Dashboard central + Configuration + file d'approbations unifiée ; CRM tout-en-un ; mobile 4 profils	6, 7, 8
Fiches M1-M8 avec règles d'or ; fondations M8 arrêtées ; findings recherche M7 intégrés	9
Ultra-sécurisé ; Loi 25 produit ; mention professionnelle fiscale	14 ; REG-10
Passe 01 : sécurité agentique, quorum de consensus, contrats exécutables, observabilité OTel, rétention fiscale 6 ans, français Loi 96, coupe-circuits de coût	REG-13 ; 4, 10.2, 11, 14 ; Annexes A-C
Passe 02 : stack mobile + sync hors-ligne serveur-autoritaire, outbox + idempotence, migrations expand-contract, PITR + restauration prouvée, IdP identité	8 ; 4 ; 11 ; 14.1 ; Annexe D
Passe 03 : pile vocale M1 chiffrée, faisabilité vision M3 quantifiée, standard API (RFC 9457 + idempotence + backoff), threat modeling 4 questions/STRIDE/LINDDUN, n8n borné, déploiement Compose	9 ; 11 ; 14.1 ; Annexe E
Passe 04 : pipeline RAG hybride+rerank chiffré, gates d'éval LLM en CI (goldens), rédaction PII avant cloud, sorties structurées anti-fabrication, accessibilité AA, hygiène des tables	13 ; 11 ; 14 ; 8 ; Annexe F
Passe 05 : chaîne d'approvisionnement durcie (SHA, lockfiles gelés, cooldown, provenance), sandbox micro-VM du code agent, runbook incident, adoption terrain, AUDIT DE COHÉRENCE INTERNE du plan (2 dérives corrigées + vérification par script)	12 ; 14.1 ; Annexe G
Directive structurelle Nathan : dossier « Modules M » + M1..M10 étiquetés, frameworks/ par module (un dossier par framework à son nom), exception prérequis infra hôte, GitHub exploité à 100 %	D-038 ; 5.1, 5.2, 12
Passe PERFECTION : pré-mortem Klein (8 causes -> 8 correctifs), walkthrough story pilote (5 frictions fermées), stress-test Phase 0 (Phase C + réordonnancement + marquage bloquant), installation vierge (profils D-043), calibration METR des stories (D-044), hooks Claude Code vérifiés mot pour mot	D-043..D-045 ; 10bis ; 16 ; Annexe I

Annexe A — Décisions actées

ID	Décision
D-001	GitHub retenu comme plateforme, plein potentiel du début à la fin
D-002	BMAD v6 = cadre d'exécution — AMENDÉE PAR D-039 (BMAD + TDAD + Superpowers par niveaux ; jamais deux méthodologies lourdes sur la même story)
D-003	Roster initial — REMPLACÉE PAR D-040 (équipe PROOF STACK §2 : Sentinelle DeepSeek+GLM, reviewers 3x3, Canon Keeper Opus+gpt-5.5)
D-004	Strangler Fig pour l'existant ; caractérisation avant raccord

ID	Décision
D-005	Core autonome clé en main : briques embarquées, README, mode bridé horaire
D-006	Briques candidates jamais figées : surclassement automatique via Q1-Q6 + wrapper
D-007	Plans = QUOI ; contexte d'implémentation antérieur périmé ; adaptation au consensus
D-008	M7 Marketing & Croissance IA = PREMIER module en développement (ratifié Nathan) ; build par vagues, dépendances GATED derrière contrats gelés
D-009	Activation briques M7 : gratuit toujours ON ; payant OFF par défaut sauf Local Falcon (ratifié Nathan, juin 2026)
D-010	Services transversaux du core : Approval Engine, autorisation fine (candidat OpenFGA), audit/ledger, Gate Engine, bus de notifications, registre de métriques — une mécanique, huit consommateurs
D-011	Contrat M10 gelé en Phase 0 + shim mémoire minimal dans le core ; M10 complet remplacera le shim sans rien casser
D-012	MMM par défaut à l'échelle GRS : PyMC-Marketing (CPU, RAM-light) ; Meridian = alternative documentée (GPU, Scenario Planner, données Google) ; propriété M8, consommation M7.11
D-013	grs-seo-knowledge porte les positions GEO par surface : Google = SEO classique (llms.txt ignoré par Google) ; assistants IA = tactiques Princeton (citations/stats/quotes) + consensus NAP + crawlabilité bots IA
D-014	Contrat mémoire M10 réel gelé en Phase 0 : event log immuable, context packs, registres (décisions, prompts, plans), Preflight/Postflight ; le shim du core en implémente le noyau ; Preflight (aucun agent ne code sans contexte) et Postflight (tout travail produit une trace) = loi du harnais pour TOUS les agents dès la Phase 0
D-015	Contrats d'interface gelés = contrats exécutables dirigés par le consommateur (Pact) ; broker versionné ; gate can-i-deploy intégré à G4 ; le module construit plus tard doit satisfaire le pact
D-016	Observabilité = 8e service transversal : OpenTelemetry + conventions sémantiques GenAI sur toutes les traces LLM/agents ; backend candidat Langfuse self-host (OTLP natif) ; versions épinglées tant que les conventions sont expérimentales
D-017	Protocole de dissidence du consensus : unanimité visée ; blocage persistant après un débat ciblé = arbitrage Nathan avec dissidence documentée au ledger ; re-vote limité aux sections modifiées
D-018	Mobile : React Native + Expo (candidat no 1, TS partagé, OTA) ; sync hors-ligne PowerSync Open Edition self-host — lectures partitionnées par Sync Rules (RBAC au niveau sync), écritures via l'API GRS (server-authoritative, gates préservés) ; pas de CRDT ; alternatives ElectricSQL / WatermelonDB / Flutter documentées
D-019	Fiabilité événementielle : outbox transactionnel (événement écrit dans la même transaction que la donnée) + relais + consommateurs idempotents (inbox/event_id) + DLQ ; obligatoire sur toute cascade inter-modules, cas canonique = clôture M4
D-020	Migrations : expand-contract obligatoire (3 déploiements, backfill par lots, contract jamais pressé) ; migrations = code revu et testé sur volume prod ; changement de schéma = approbation distincte ; bundle de migrations par release clé-en-main
D-021	Sauvegarde : pgBackRest full+diff+WAL => PITR ; dépôt hors hôte chiffré ; 3-2-1 ; RPO <= 15 min ; RTO = durée mesurée du dernier exercice ; restauration automatisée hebdomadaire prouvée au ledger ; échec de restauration = Sev1
D-022	Identité : Zitadel candidat no 1 (Go, ~100 MB RAM, event-sourcé/audit natif, multi-tenant, WebAuthn/passkeys) en service séparé (AGPL contenue) apparié à OpenFGA pour l'autorisation ressource ; Authentik/Keycloak documentés
D-023	Voix M1 : Pipecat candidat no 1 (BSD-2, v1.0, Flows = phases 0->5, OTel natif) ; LiveKit Agents alternative (SIP, 750-900 ms) ; cibles <= 800 ms p50 perçu / <= 1,5 s p95 ; STT/TTS locaux candidats, cloud gated ; VAD + barge-in dès phase 2
D-024	Vision M3 : hiérarchie de précision imposée — référence d'échelle > multi-vues > mono à bornes larges ; preuve : MAE 0,454 m au MIEUX en extérieur sans référence ; briques Depth Pro / Depth Anything V2 / Metric3D v2 ; guidage de capture dans l'app vendeur = le levier
D-025	API surfaces : OpenAPI + RFC 9457 (erreurs) + Idempotency-Key (POST non sûrs) + backoff exponentiel avec jitter et plafond + /v1 + pagination curseur ; Playwright et k6 candidats
D-026	Threat modeling par module : quatre questions + STRIDE (technique) + LINDDUN (vie privée Loi 25) au PRD/Architecture ; definition-of-done légère ; registre de menaces au ledger ; red team G2
D-027	n8n = brique optionnelle bornée (jamais payer/supprimer/masquer/engager), jamais sur le chemin critique ; le planificateur du core = autorité
D-028	Clé-en-main : Docker Compose mono-noeud par défaut ; profil multi-noeuds documenté

ID	Décision
D-029	RAG par collection : réécriture -> hybride (dense+BM25) fusion RRF -> rerank cross-encoder -> top 5-8 avec citations ; preuve Recall@5 0,816 vs 0,587 dense seul ; BM25 domine sur documents financiers ; nuance : mesurer par collection, l'hybride peut dégrader les meilleurs embedders ; ré-indexation versionnée
D-030	Gates d'éval LLM en CI : goldens versionnés sourcés des échecs réels, régression vs baseline = build rouge ; DeepEval/Ragas/Promptfoo (gating) + Langfuse (suivi) ; Promptfoo = surclassement de modèles (REG-08 étendu) ; changement de prompt = PR avec run d'éval au ledger
D-031	Rédaction PII obligatoire avant tout cloud/journal/trace : Presidio candidat (texte, image, tabulaire, opérateurs réversibles) ; rédaction des CHUNKS RAG incluse ; mapping réversible sous clé locale ; opérationnalise l'EFVP « cloud minimisé »
D-032	Sorties structurées 3 couches (schéma -> décodage contraint -> validation post-parse) ; règle d'or : échappatoire obligatoire sur tout enum (unknown/other), champs absents nullables jamais required — un schéma sans issue fabrique l'hallucination
D-033	Surfaces dashboard + mobile : WCAG 2.2 AA minimum (site public reste AAA) ; garde-fous automatisés
D-034	Tables à forte croissance : partitionnement temporel + archivage froid + rétention par classe + PgBouncer
D-035	Chaîne d'approvisionnement : Actions épinglées au SHA complet ; lockfiles + installations gelées ; ignore-scripts par défaut ; cooldown de versions ; token moindre privilège + CODEOWNERS workflows ; SBOM + provenance (origine, pas innocuité) ; hash SHA-256 des briques/poids vérifié ; LiteLLM épinglé par hash (compromis PyPI mars 2026)
D-036	Sandbox d'exécution : micro-VM (Firecracker/Kata) pour code agent/non fiable/briques en essai ; gVisor intermédiaire ; conteneurs durcis pour code revu seulement ; réseau sortant en liste blanche ; secrets jamais montés
D-037	Réponse à incident : runbook versionné (isoler -> rotation -> restaurer -> notifier -> post-mortem sans blâme au ledger) ; exercice annuel
D-038	RATIFIÉE NATHAN — Arborescence : dossier « Modules M » contenant M1..M10 clairement étiquetés ; chaque module a frameworks/ avec UN dossier par framework au nom exact ; exception = prérequis de l'infrastructure hôte (C1, jamais dans un module) ; partagé 2+ modules = core/frameworks/ sans duplication (complément à ratifier) ; lint de structure en check requis ; GitHub exploité à 100 % (CODEOWNERS/labels/milestones/releases par module)
D-039	RATIFIÉE NATHAN — PROOF STACK v1.1 = méthode d'exécution officielle, exécutée intégralement avec Claude Code ; profil PROOF-Team (repli Solo annoncé) ; PREUVE:/BLOCKED: à chaque étape ; structuration BMAD + TDAD + Superpowers pinnée par niveaux
D-040	RATIFIÉE NATHAN — Équipe officielle = table PROOF STACK §2 (remplace 10.1) : Orchestrateur ne se relit jamais ; Kimi K2.7 missions longues ; Kimi K3 stories difficiles avec titularisation par firing test ; DeepSeek bulk ; Grok parallèle worktrees ; reviewers aveugles 3x3 DeepSeek·Gemini·LongCat ; pool Kimi·Qwen·MiMo ; Sentinelle DeepSeek+GLM (rouvre, ne valide jamais) ; Canon Keeper Opus+gpt-5.5 (vendor OpenAI ajouté)
D-041	RATIFIÉE NATHAN — Pipeline par story A->G (pack scopé -> TDD réel -> gates locaux -> PR/CI -> revue aveugle 3x3 scellée simultanée -> dépouillement PAR SCRIPT jamais un LLM -> merge registré) + gate de fin de phase v1.1 (e2e réel + 3x3 du rapport + signature sur pièces) ; remplace le mapping G0-G5
D-042	RATIFIÉE NATHAN — Registres = 4 jsonl append-only HASH-CHAÎNÉS via ledger_append.py, jamais à la main ; 12 briques de méthode au manifest Phase 0 avec preuve d'activation ; interdits étendus bloqués par hooks (CLAUDE_CODE_SUBAGENT_MODEL global, compression code/diffs/packs, mocks tests-only)
D-043	Deux profils d’installation clé-en-main : « infra-existante » (branchement) et « starter » (passerelle + vectordb + PostgreSQL auto-déployés en conteneurs — le RAG lui-même devient clé en main) ; wizard premier démarrage (admin+MFA, français, mode horaire par défaut jusqu’à preuve du temps réel)
D-044	Gate de taille de story calibré METR : estimation humain-équivalent obligatoire ; cible <= 1 h ; cap dur 3 h (zone 80 % de fiabilité) ; > 3 h = découpage obligatoire ; taux d’échec par tranche de durée = métrique de gouvernance qui ajuste le cap par les données
D-045	Phase C (consensus outillé par consensus_run.py = brique de méthode no 13) précède la Phase 0 ; Phase 0 réordonnée par analyse de dépendances avec marquage BLOQUANT-STORY-1 vs PARALLÉLISABLE ; 16 correctifs de simulation C-01..C-16 intégrés (Annexe I)

Annexe B — Risques ouverts

ID	Risque	Mitigation
R-001	Corrélation vendeur Composer / Grok (xAI)	Jamais deux avis indépendants, jamais appariés
R-002	Clé passerelle placeholder sur routes facturées	Rotation Phase 0 (P0-4) — bloquant

ID	Risque	Mitigation
R-003	Auto-préférence des juges LLM	Familles croisées, obfuscation, dimensions forcées, panel
R-004	Agent Teams expérimental	Ledger = vérité ; tâches bornées ; commits fréquents
R-005	Géoloc employés, enregistrements, session replay = zones Loi 25	EFVP + consentement + collecte limitée + rétention
R-006	Conflit manuel (CRM) vs automatique (GRS)	Base unique + journalisation + priorités par module
R-007	Briques GPL/AGPL vs vision SaaS	Gate Q1 ; AGPL = services séparés appelés par API
R-008	Dépendances inter-modules denses	Contrats gelés + mocks GATED ; ordre au consensus
R-009	M5 construit trop tôt = projections sur du vide	M5 après données réelles ; ordre à ratifier
R-010	M10 = dépendance universelle (RÉSOLU : plan reçu)	D-014 : contrat réel gelé ; shim Phase 0 = noyau du contrat
R-011	Prolifération de mécanismes dupliqués (6 systèmes d'approbation...)	D-010 : services transversaux du core
R-012	PostHog stack lourde (min 4 vCPU/16GB) vs RAM rare	Noeud avec marge OU report du CRO à une vague tardive ; réévaluer alternatives légères au consensus
R-013	Approbation API Google (GBP) = délai externe de quelques jours à une semaine	Demander l'accès dès la Phase 0, avant la vague 1 de M7
R-014	Coût API X variable (changé deux fois depuis fév. 2026)	Budget surveillé par le CMO Advisor, jamais présumé fixe
R-015	Injection de prompt indirecte via M1 (courriels), M7 (web), M6 (fournisseurs) — risque no 1 OWASP, précédent réel EchoLeak	REG-13 : défense en profondeur + provenance + gates + red team Grok 4.5 + mode ombre
R-016	Tension rétention fiscale 6 ans vs minimisation Loi 25	Calendrier par classe : fiscal = exception légale documentée ; classes sensibles = durées courtes
R-017	Conventions OTel GenAI encore expérimentales (instabilité de schéma)	Versions épinglées ; double émission si migration ; Langfuse mappe les deux
R-018	Conflits de sync hors-ligne (même donnée modifiée offline et au serveur)	Serveur-autoritaire ; données sensibles = pending_review avec les deux versions ; jamais d'écrasement silencieux
R-019	Moteurs de sync jeunes (PowerSync/Electric : API et Sync Rules mouvantes ; complexité des règles à l'échelle)	Wrapper d'interface (REG-08), version épinglée, tests des Sync Rules par scénario utilisateur, WatermelonDB en repli
R-020	Punch hors-ligne = vecteur de triche potentiel (horloge/GPS falsifiables)	Événement signé + fix GPS au moment du tap + attestation appareil + validation serveur + marquage OFFLINE + revue admin sans accusation auto
R-021	Latence vocale : les médianes industrielles réelles (1,4-1,7 s, p99 3-5 s) sont loin du seuil de 800 ms	Pipeline streaming bout-en-bout, budgets par étage mesurés au p95, modèles locaux rapides, phases M1 progressives — la phase 4 n'ouvre pas si le p95 n'est pas tenu
R-022	Mesure photo sans référence d'échelle : ~0,45 m d'erreur moyenne AU MIEUX en extérieur	D-024 : prix final bloqué sans référence d'échelle ou multi-vues ; guidage de capture ; bornes affichées toujours
R-023	Idempotency-Key encore à l'état de brouillon IETF (spec mouvante)	Implémenter le pattern (clé + table de déduplication), suivre la spec, version épinglée du comportement documentée
R-024	SW1 = point de défaillance unique public (portail client down si SW1 tombe)	Acceptation documentée v1 + page de statut + restauration rapide prouvée (P0-9) ; haute disponibilité différée et documentée
R-025	Schémas de sortie sans échappatoire = fabrication structurelle silencieuse	Règle D-032 systématique + lint de schémas en CI (tout enum sans valeur d'échappement = build rouge)

ID	Risque	Mitigation
R-026	Changement de modèle d'embedding = index vectoriel silencieusement invalide	Migration d'index complète versionnée + évaluation sur goldens (D-030) avant bascule + les deux index en parallèle pendant la transition
R-027	Adoption terrain : résistance humaine au punch géorepéré et aux formulaires de clôture	Pilote restreint (1-2 employés), co-conception, formation, période de rodage SANS sanction, feedback dans l'app ; taux d'adoption = métrique canonique au registre
R-028	Chaîne d'approvisionnement : notre propre passerelle (LiteLLM) figure parmi les paquets compromis de la vague 2026	D-035 intégral ; LiteLLM épinglé version+hash, cooldown, vérification à chaque installation
R-029	Évasion de sandbox par code généré (noyau partagé insuffisant)	D-036 : micro-VM par défaut pour tout code non revu ; conteneur durci = privilège, pas défaut
R-030	Les scripts déterministes de la méthode (ledger_append, civ_review, tally, hooks) deviennent des points uniques de confiance	Méthodologie-as-code : versionnés, testés, revus comme du code de premier ordre ; amendement = PR + circuit de revue ; leur propre suite de tests au registre
R-031	Goulot humain : toutes les validations T3/approbations convergent vers Nathan seul	C-04 : budget de validation quotidien, file priorisée, digest, SLA interne journalisé
R-032	Coût du protocole : ~10+ appels modèle par story (3x3 + tandems + orchestrateur)	Budgets D-016/coupe-circuits ; proposition Q9 : classe « story triviale » (docs/config-only) à profil réduit ANNONCÉ — à ratifier Nathan
R-033	Stop-hook/turn-guard mal configuré = boucle infinie documentée	C-08 : condition de sortie + compteur obligatoires sur tout hook bloquant

Annexe C — Changelog Passe 01 (chaque modification -> sa preuve)

Modification	Preuve / source
REG-13 + politique de sécurité agentique (14.1) + mode ombre	OWASP LLM Top 10 2025 (LLM01 = injection, la plus exploitée ; défense en profondeur, ni RAG ni fine-tuning ne suffisent) ; OWASP Top 10 Agentic 2026 (ASI01) ; EchoLeak = premier exploit zéro-clic réel en production ; spotlighting (Hines et al. 2024)
D-015 : contrats gelés -> pacts exécutables + can-i-deploy à G4	Pact (contrats consommateur-pilotés = artefacts versionnés ; vérification sans co-exécution des services ; broker = source de vérité déploiement) ; caveat intégré : justifié seulement si plusieurs consommateurs — cas GRS
D-016 : observabilité = 8e service transversal (OTel GenAI + Langfuse)	OTel = standard CNCF neutre, substrat par défaut de l'observabilité LLM 2026 ; espace gen_ai.* (modèle, tokens, coûts, outils) ; Langfuse conforme + endpoint OTLP ; statut expérimental (R-017)
D-017 : quorum et dissidence du consensus	Faiblesse de conception relevée à l'audit interne : « consensus de tous » sans règle de blocage = veto infini possible. Correction de protocole ; aucune source externe requise
Rétention fiscale 6 ans par classe (14.2)	Revenu Québec (pages officielles, confirmé sur 5 pages distinctes) : 6 ans après la dernière année visée ; support électronique identique + intégrité ; destruction anticipée = autorisation écrite
Français d'abord — Loi 96 (14.2)	Analyses juridiques 2025-2026 : logiciels/sites/apps offerts aux clients QC accessibles en français à conditions équivalentes ; contrats d'adhésion en français ; francisation OQLF 25+ employés (juin 2025) ; sanctions 700 $-90 000 $
Coupe-circuits de coût LLM (14.1)	OWASP agents 2026 : boucles agentiques = risque financier structurel ; capacité budgets-par-clé de la passerelle marquée A CONFIRMER en P0-4 (hedge honnête, pas de fake done)

Annexe D — Changelog Passe 02 (chaque modification -> sa preuve)

Modification	Preuve / source
D-018 : PowerSync + RN/Expo, écritures server-authoritative, pas de CRDT	Revues 2026 : PowerSync = sync bidirectionnelle la plus mature pour mobile en production, Open Edition self-host gratuite, outils de terrain = cas d'usage nommé ; « ne pas importer la complexité CRDT pour un problème qu'on n'a pas — la plupart des apps ont juste besoin d'écritures en file »
D-019 : outbox transactionnel + idempotence	Littérature du double-écrit (AWS Prescriptive Guidance, analyses 2025-2026) : at-least-once => consommateurs idempotents obligatoires ; exemple d'échec documenté = facture émise en double ; FOR UPDATE SKIP LOCKED / réplication logique pour le relais
D-020 : expand-contract	Guides 2026 convergents : jamais schéma+code dans le même pas ; 3 déploiements ; backfill par lots bornés (verrous) ; contract = seule étape irréversible, jamais pressée ; tester sur volume production
D-021 : pgBackRest PITR + restauration prouvée	pgBackRest (full+diff+WAL, PITR à la seconde, chiffrement aes-256-cbc) ; « une sauvegarde jamais testée n'est pas une sauvegarde — automatiser la vérification hebdomadaire » ; « le RTO réel = la durée enregistrée du test, pas le chiffre de la doc » ; « échec du test de restauration = Sev1 » ; caveat rejeu WAL glouton
D-022 : Zitadel + appairage PDP	Comparatifs 2026 : Zitadel Go ~100 MB vs Authentik ~300 MB ; event-sourcé = auditabilité ; multi-tenant natif = fit B2B/SaaS ; AGPL 3.0 depuis 2025 ; « ni Keycloak ni Zitadel ne sont la couche d'autorisation — appairer à un PDP ressource-niveau »

Annexe E — Changelog Passe 03 (chaque modification -> sa preuve)

Modification	Preuve / source
D-023 : Pipecat/LiveKit + budgets vocaux recalibrés	Comparatifs 2026 : Pipecat v1.0 avril 2026, BSD-2, 60+ intégrations, Flows, OTel natif ; LiveKit 750-900 ms vs Pipecat 800-950 ms (écart souvent négligeable) ; seuil 800 ms = fin de l'effet robotique ; médianes production 1,4-1,7 s, p99 3-5 s ; « mesurer au p95, pas à la moyenne »
D-024 : hiérarchie de précision vision + chiffres	Depth Pro (arXiv/OpenReview) : métrique sans intrinsèques, 2,25 MP en 0,3 s, SOTA zero-shot ; banc extérieur avec vérité terrain ChARUCO : Depth Anything V2 MAE 0,454 m (corr. 0,962), ZoeDepth 3,09 m — le demi-mètre d'erreur justifie le blocage de prix sans référence d'échelle
D-025 : RFC 9457 + Idempotency-Key + backoff	RFC 9457 = IETF Standards Track (juillet 2023, remplace 7807) ; brouillon IETF Standards Track pour l'en-tête d'idempotence (pattern Stripe) ; Amazon Builders' Library : 3 retries x 3 couches = amplification 243x
D-026 : quatre questions + STRIDE + LINDDUN	Threat Modeling Manifesto / Shostack (quatre questions) ; OWASP Cheat Sheet (STRIDE mature, flexible) ; STRIDE choisi en littérature pour son faible taux de faux positifs ; LINDDUN = cadre privacy inspiré de STRIDE (MDN) ; garde-fou Shostack : DoD légère sinon gaming
D-027 / D-028 : n8n borné ; Compose par défaut	Décisions de conception : D-027 découle des règles déjà présentes dans les dix plans (invariants) ; D-028 = arbitrage RAM/simplicité documenté, profil multi-noeuds préservé — à ratifier au consensus

Annexe F — Changelog Passe 04 (chaque modification -> sa preuve)

Modification	Preuve / source
D-029 : pipeline RAG hybride + rerank	T2-RAGBench (EACL 2026, 23 088 requêtes, documents financiers) : hybride+rerank Recall@5 0,816 vs 0,695 (RRF seul) vs 0,587 (dense seul), p<0,001 ; BM25 > dense sur financier ; +7,4 % NDCG sur WANDS ; contre-exemple SemEval (BM25 peut dégrader les meilleurs embedders) -> mesurer
D-030 : goldens + gates CI	DeepEval (assert_test en CI : « les scores qui échouent cassent le build ») ; guides golden datasets 2026 (versionnés, sourcés des échecs, baseline + tolérance) ; pattern deux-outils (cadre CI léger + plateforme) ; Promptfoo = comparaison inter-modèles avant bascule
D-031 : rédaction PII avant cloud	Presidio (Microsoft, open-source : NER+regex+checksums, image/structured, opérateurs réversibles) ; pipeline avec rédaction des chunks (« piège courant : les documents récupérés contiennent plus de PII que le message ») ; PAPILLON : fuite 100 % -> 7,5 % avec filtre local avant API distante ; limites (niveau message, cohérence) documentées
D-032 : anti-fabrication structurée	« L'enum sans échappatoire est le mécanisme par lequel un schéma rigide fabrique l'hallucination » (règle d'échappement + nullable) ; Park et al. NeurIPS 2024 (distorsion de distribution) ; « structure snowballing » (arXiv 2026) ; NAACL 2024 : ancrer puis contraindre se cumulent ; pipeline 3 couches (Pydantic -> contraintes -> validation)
D-033 / D-034 : accessibilité AA ; hygiène des tables	Décisions normatives internes cohérentes (AAA déjà engagé sur le site public ; pratiques standard de partitionnement/pooling) — à ratifier au consensus

Annexe G — Changelog Passe 05 + audit de cohérence interne

Modification	Preuve / source
D-035 : durcissement chaîne d'approvisionnement	Vague 2026 documentée : Trivy, axios, LiteLLM (PyPI, mars 2026), TanStack (84 versions en 6 min AVEC provenance SLSA L3 valide), tag-hijack d'action ; « les tags sont mutables, seul le SHA de commit est immuable » (GitHub) ; npm ci échoue sur divergence ; cooldowns devenus défauts d'écosystème ; « une signature prouve l'origine, pas l'innocuité »
D-036 : sandbox micro-VM du code agent	Consensus praticiens 2026 : « Docker/runc n'est pas suffisant pour du code généré par IA » ; Firecracker ~125 ms / <5 MiB / snapshot-restore 5-30 ms ; gVisor bloque le GPU passthrough, Firecracker/VFIO le permet ; ~90 % d'incidents en moins (source unique, indicatif)
D-037 : runbook incident ; R-027 : adoption terrain	Décisions de processus internes (pratiques standard IR ; gestion du changement) — à ratifier au consensus ; R-027 découle du risque humain réel des fonctions anti-triche
AUDIT DE COHÉRENCE : « 7 services » -> 8 (table C2, dérive de la Passe 01 qui a ajouté l'observabilité en section 4 sans mettre à jour l'architecture)	Incohérence détectée par relecture croisée ; corrigée ; c'est exactement la dérive fichier-doc que REG-12 interdit — le plan subit sa propre doctrine
AUDIT DE COHÉRENCE : « les huit plans » -> « les dix plans » (section 4, texte écrit quand 8/10 plans étaient reçus)	Incohérence temporelle détectée et corrigée ; vérification finale par script : présence de D-001..D-037, R-001..R-029, REG-01..REG-13, absence des anciennes formulations — sortie du script = preuve au ledger
D-038 : arborescence « Modules M » / frameworks par module / exception infra hôte / GitHub 100 %	Directive DIRECTE de Nathan (2026-07-16), ratifiée à réception ; le cas partagé (core/frameworks, anti-duplication) est le seul complément proposé, à ratifier ; enforcement mécanique = bijection par module en check requis

Annexe H — Intégration PROOF STACK (fusion finale)

Élément	Traçabilité
D-039..D-042 : méthode, équipe, pipeline A->G, registres hash-chaînés	Directive DIRECTE Nathan : TEMPLATE-PROJET-PROOF-STACK-COMPLET-v1-20260716 (lu intégralement, 10 sections) ; ratifiées à réception
Apports nets au squelette : tally par script (aucun LLM n'écrit un score) ; verdicts scellés simultanés ; hash-chaînage (l'injection PHASE_ACCEPTED devient impossible, plus seulement interdite) ; firing tests + titularisation ; digest <=400 tokens par hook ; Sentinelle qui rouvre sans valider ; méthodologie-as-code	Sections 10bis §1-§8 ; convergence totale avec REG-01..05, REJ-0040, 8.4
Amendements : D-002 -> D-039 (méthodologies par niveaux) ; D-003 -> D-040 (équipe §2) ; G0-G5 -> A->G + fin de phase v1.1 (D-041)	Marqués dans l'Annexe A ; anciennes formulations conservées pour traçabilité
Profil PROOF-Team proposé (3+ vendors disponibles) ; gpt-5.5 ajouté au roster (vendor OpenAI, décorrélation accrue)	À confirmer au PACK v2 (Q8) ; accès gpt-5.5 vérifié en calibration §0 avec preuve de route

Annexe I — Passe PERFECTION : simulations internes (preuves)

Simulation / élément	Résultat et preuve
SIM-1 PRÉ-MORTEM (protocole Klein : « GRS est mort en juillet 2027 — pourquoi ? », causes écrites indépendamment puis classées)	8 causes -> 8 correctifs : Phase 0 infinie (C-01/C-14 marquage bloquant), consensus manuel 12 modèles (C-02 consensus_run.py), stories trop grosses (C-03/D-044), goulot Nathan (C-04), coût du protocole (C-05/Q9), doc:none par défaut (C-06), environnement hétérogène (C-07 Ubuntu/WSL2), Stop-hook infini (C-08). Fondement : l'hindsight prospectif augmente de 30 % l'identification correcte des causes (Mitchell/Russo/Pennington 1989 ; Klein, HBR 2007)
SIM-2 WALKTHROUGH complet de M7-S000 dans le pipeline A->G, étape par étape	5 frictions trouvées AVANT la vraie vie : bootstrap sans retrieval (C-09), exclusions du no-stub-scan versionnées (C-10), harnais e2e requis dès la story 1 (C-11), exception verdicts sans échappatoire documentée (C-12), story pilote définie en mode ombre <= 1 h (C-13)
SIM-3 STRESS-TEST des dépendances Phase 0 (graphe complet P0-1..P0-10 + briques de méthode)	1 inversion trouvée (rotation de clé AVANT le branchement des routes), Phase C extraite en préalable, marquage BLOQUANT-STORY-1 vs PARALLÉLISABLE, sauvegardes bloquantes avant la première donnée de production ; zéro dépendance circulaire (C-14)
SIM-4 INSTALLATION sur infra vierge (persona : PME, un serveur Ubuntu nu)	Gap fermé : le prérequis passerelle/RAG bloquait le clé-en-main réel -> profil « starter » auto-déployé (D-043, C-15) + wizard premier démarrage avec mode horaire par défaut jusqu'à preuve du temps réel (C-16)
CALIBRATION METR (D-044)	Mesures mai 2026 : ~100 % de réussite < 4 min humain, < 10 % > 4 h ; la durée = prédicteur no 1 ; doubler la durée quadruple l'échec ; horizon 80 % ~ 1/4 de l'horizon 50 % (top : 14 h 30 à 50 %) ; effondrement en contexte long (40-50 % -> < 10 %) valide le contexte propre par story
HOOKS CLAUDE CODE vérifiés contre la doc officielle	UserPromptSubmit / SessionStart : stdout injecté au contexte (mécanisme exact du digest) ; exit 2 = blocage ; PreCompact + SessionStart source=compact confirmés MOT POUR MOT (parité post-compaction de PROOF STACK) ; settings.json versionnable au projet ; piège documenté du Stop-hook toujours-bloquant -> C-08

Prochaine action : PERFECTION v1.1 SCELLÉE (16 correctifs simulés et prouvés) -> PHASE C : consensus outillé (consensus_run.py) -> PACK v2.1 (Q7 licence, Q8 profil, Q9 stories triviales) -> distribution aux 12 modèles -> revues en aveugle -> ratification Nathan (rôles, briques, trous M7, ordre des modules suivants) -> Phase 0 -> GO M7 vague 1.